LUCIO VARGAS DE ALBUQUERQUE NUNES

OTIMIZAÇÃO DA MONTAGEM DE ELENCO NO FUTEBOL: APRENDIZADO DE MÁQUINA PARA REDUÇÃO DA IMPREVISIBILIDADE EM CONTRATAÇÕES

Exame de Qualificação

LAVRAS – MG

2025

LUCIO VARGAS DE ALBUQUERQUE NUNES

OTIMIZAÇÃO DA MONTAGEM DE ELENCO NO FUTEBOL:
APRENDIZADO DE MÁQUINA PARA REDUÇÃO DA
IMPREVISIBILIDADE EM CONTRATAÇÕES
Exame de Qualificação

Tese apresentada à Universidade Federal de Lavras, como parte das exigências do Programa de Pós-Graduação em Ciência da Computação, para a obtenção do título de Mestre.

Prof. Dr. Dilson Lucas Pereira
Orientador

LAVRAS – MG
2025

LUCIO VARGAS DE ALBUQUERQUE NUNES

OTIMIZAÇÃO DA MONTAGEM DE ELENCO NO FUTEBOL:
APRENDIZADO DE MÁQUINA PARA REDUÇÃO DA
IMPREVISIBILIDADE EM CONTRATAÇÕES
OPTIMIZATION OF SQUAD BUILDING IN FOOTBALL: MACHINE
LEARNING FOR REDUCING UNCERTAINTY IN PLAYER RECRUITMENT

Tese apresentada à Universidade Federal de Lavras, como parte das exigências do Programa de Pós-Graduação em Ciência da Computação, para a obtenção do título de Mestre.

APROVADA em 23 de Março de 2025.

Prof. Dr. Mayron César de Oliveira Moreira	UFLA
Prof. Dr. Cristiano Arbex Valle	UFMG
Profa. Dra. Elaine Cecília Gatto	UFLA

Prof. Dr. Dilson Lucas Pereira
Orientador

LAVRAS – MG
2025

RESUMO

O futebol, o esporte mais popular do mundo, é marcado por sua alta imprevisibilidade, o que dificulta a montagem de elencos, as escolhas táticas e a seleção de jogadores para cada partida. Este estudo explora o uso de técnicas de Aprendizado de Máquina (AM) e Pesquisa Operacional (PO) como ferramentas de apoio à tomada de decisão no contexto esportivo. O objetivo é desenvolver um modelo preditivo capaz de estimar o desempenho dos clubes com base em dados estatísticos individuais dos jogadores. A partir dessas previsões, são aplicados algoritmos de otimização para orientar estratégias de contratações, permitindo decisões mais eficientes e fundamentadas em dados. Essa abordagem visa reduzir a subjetividade nos processos de gestão esportiva, promovendo maior assertividade nas decisões de mercado por parte dos clubes.

Palavras-chave: Futebol. Aprendizado de Máquina. Elenco.

ABSTRACT

Football, the most popular sport in the world, is marked by a high degree of unpredictability, which makes squad building, tactical planning, and player selection particularly challenging. This study explores the use of Machine Learning (ML) and Operations Research (OR) techniques as decision-support tools in the sports context. The objective is to develop predictive models capable of estimating club performance based on individual player statistics. These predictions are then used as inputs for optimization algorithms aimed at guiding player recruitment strategies, enabling more efficient and data-driven decisions. This approach seeks to reduce subjectivity in sports management processes, promoting greater accuracy in market decisions and contributing to a more strategic, evidence-based decision-making framework within football clubs.

Keywords: Football. Machine Learning. Squad.

LISTA DE FIGURAS

Figura 3.1 – Pipeline CRISP-DM 18
Figura 3.2 – AD para Iris 20
Figura 5.1 – Escolha do Número Ideal de Componentes do PCA 46
Figura 6.1 – Relação entre pontuações reais e previstas pelo modelo MLP na Premier League. 63

SUMÁRIO

1 INTRODUÇÃO 11
2 OBJETIVOS 15
2.1 Objetivo Geral 15
2.2 Objetivos Específicos 16
3 REFERENCIAL TEÓRICO 17
3.1 Aprendizado de Máquina 17
3.1.1 Árvore de Decisão (AD) 19
3.1.2 Random Forest (RF) 21
3.1.3 Support Vector Machine (SVM) 21
3.1.4 Gradient Boosting (XGBoost) 22
3.1.5 Multilayer Perceptron (MLP) 22
3.1.6 Generalização, Viés e Variância 23
3.1.7 Seleção de Atributos e Redução de Dimensionalidade 24
3.2 Pesquisa Operacional 24
3.2.1 Algoritmo Guloso 25
3.2.2 Algoritmo de Busca Local 28
4 REVISÃO DE LITERATURA 31
4.1 Aprendizado de Máquina no contexto esportivo 31
4.1.1 Previsão de resultados de partidas 32
4.1.2 Aplicações recentes do AM no futebol 34
4.1.3 Desempenho agregado e determinantes de temporada 35
4.2 Pesquisa Operacional no contexto esportivo 37
4.2.1 Otimização Prescritiva e Problemas Combinatórios no Esporte 39
5 METODOLOGIA 41
5.1 Coleta dos Dados 41
5.2 Pré-processamento dos Dados 42
5.2.1 Base de Dados dos Jogadores 42

5.2.1.1 **Principal Component Analysis (PCA)** 46
5.2.2 **Base de Dados dos Jogos** 47
5.3 **Treinamento do Modelo de AM** 47
5.3.1 **Modelo de Aprendizado de Máquina** 48
5.4 **Otimização** 50
5.4.1 **Cálculo do Valor de Mercado Sintético** 50
5.4.2 **Algoritmos Gulosos para Seleção Iterativa de Jogadores** . . . 52
5.4.3 **Algoritmos de Busca Local para Otimização do Elenco** 54
6 **RESULTADOS E DISCUSSÕES** 61
6.1 **Modelos de Aprendizado de Máquinas** 61
6.1.1 **Predição do Modelo MLP** 62
6.2 **Resultados da Otimização de Montagem de Elenco** 64
6.3 **Conclusões** 66
REFERÊNCIAS 67
A **Pseudoalgoritmos completos dos algoritmos gulosos** 73
B **Pseudoalgoritmos completos dos algoritmos de busca local** . . 79

1 INTRODUÇÃO

Futebol é o esporte mais popular do mundo (ALMULLA et al., 2023). Nas últimas décadas, sua evolução tecnológica e analítica tem sido notável, acompanhada por um crescimento expressivo nos lucros gerados pelas atividades esportivas (RODRIGUES; PINTO, 2023). No entanto, o futebol é um esporte altamente imprevisível. Um exemplo notável dessa imprevisibilidade foi a surpreendente conquista da English Premier League (EPL) pelo Leicester City em 2016, um feito que surpreendeu especialistas e torcedores em todo o mundo (BABOOTA; KAUR, 2019).

Dada a relevância econômica e o impacto do futebol como forma de entretenimento, pesquisadores da academia e profissionais da indústria têm se dedicado ao desafio de prever seus resultados (BABOOTA; KAUR, 2019). O crescimento do mercado de apostas reflete essa busca, impulsionando também os lucros da indústria de prognósticos esportivos (RODRIGUES; PINTO, 2023). Para se ter uma noção da magnitude econômica do futebol, entre 1º de junho e 1º de setembro de 2023, as transferências de jogadores movimentaram R$ 36,6 bilhões, conforme relatório divulgado pela Federação Internacional de Futebol (FIFA) (GLOBO Esporte, 2023). No Brasil, em 2025, a janela de transferências da Série A brasileira bateu a marca de R$ 1 bilhão em movimentações financeiras (GLOBO Esporte, 2025).

No campo da Ciência da Computação, diversos estudos têm explorado técnicas de análise de dados e estatística com o objetivo de modelar padrões e realizar predições em ambientes complexos. Um exemplo relevante é o estudo de Joseph, Fenton e Neil (2006), que investigou o uso de redes Bayesianas e outras técnicas de Aprendizado de Máquinas (AM) para prever resultados do Tottenham Hotspur entre 1995 e 1997. Outro caso notável de aplicação de dados no futebol envolve o Brighton & Hove Albion, clube inglês que, sob a gestão de Tony Bloom, implementou uma estratégia fortemente orientada por análises estatísti-

cas para recrutamento e tomada de decisões. Essa filosofia refletiu-se diretamente na política de transferências do clube, com a identificação sistemática de atletas subvalorizados no mercado. Entre os exemplos mais emblemáticos está Moisés Caicedo, contratado junto ao Independiente Del Valle em 2021 por cerca de £4,5 milhões e negociado com o Chelsea em 2023 por valores próximos a 115 milhões €, tornando-se uma das maiores vendas da história da EPL (BBC Sport, 2023; CNN Brasil, 2023). O mesmo caso de altos lucros em revendas se aplica a outros jogadores, como Alexis Mac Allister e Marc Cucurella, abordagem que permitiu que o Brighton se consolidasse como um dos clubes mais eficientes do mercado de transferências da EPL, reinvestindo os recursos obtidos e mantendo a competitividade sustentável na elite do futebol inglês (OLLEY, 2023).

Na literatura acadêmica, observa-se que a maioria dos trabalhos envolvendo AM no futebol concentra-se na previsão de resultados de partidas, probabilidades de vitória ou otimização de ganhos no contexto de apostas esportivas. Em contraste, menos estudos direcionam esforços à construção e otimização de elencos com foco na avaliação e seleção de jogadores (ABIDIN, 2021). Essa lacuna é particularmente relevante, uma vez que a escolha de um atleta para compor um elenco envolve a análise simultânea de múltiplos parâmetros técnicos, físicos e mentais, além de fatores contextuais como estilo de jogo da equipe, liga disputada e momento da carreira do jogador (ABIDIN, 2021). A natureza não determinística inerente ao futebol torna esse processo ainda mais desafiador, pois mesmo contratações que aparentam ser ideais sob múltiplos critérios podem não atingir o desempenho esperado.

Segundo Ian Graham (2024), uma transferência pode ser considerada um sucesso se o jogador iniciar 50% das partidas nas primeiras duas temporadas ("The 50% rule"). Considerando todas as transferências de mais de € 10 milhões que ocorreram na EPL entre os anos de 1992 e 2021, perto da metade (46%) não atingiram o critério estabelecido e, portanto, podem ser classificadas como fa-

lhas (GRAHAM, 2024). Entre os principais fatores associados a esses insucessos, destacam-se:

1. O jogador atual é melhor que o novo jogador;
2. O jogador não é tão bom quanto se pensava inicialmente;
3. O jogador não se encaixa no estilo do time;
4. O jogador joga fora de posição;
5. O técnico não aprova o jogador;
6. O jogador tem problemas de condicionamento físico ou pessoais.

Nos últimos anos, o Aprendizado de Máquinas (AM) tem ampliado significativamente a capacidade de análise de dados esportivos, permitindo a identificação de padrões antes imperceptíveis em grandes volumes de informação. Técnicas avançadas vêm sendo aplicadas para estimar desde a probabilidade de vitória em uma partida até o impacto de decisões táticas ao longo de uma temporada. Entretanto, tais modelos não eliminam a incerteza inerente ao futebol, atuando como ferramentas de apoio à tomada de decisão humana, com o objetivo de reduzir riscos e embasar escolhas estratégicas.

Neste contexto, o presente trabalho investiga o uso de modelos preditivos para converter estatísticas históricas de desempenho individual em estimativas de desempenho esperado de jogadores quando inseridos em contextos específicos de equipe e temporada. Diferentemente de abordagens voltadas à previsão direta de resultados de partidas ou à avaliação de talento de forma isolada, a proposta concentra-se na análise contextualizada do jogador como parte de um sistema coletivo. As previsões geradas são utilizadas como subsídio em um processo de simulação e avaliação de cenários de contratação, visando apoiar decisões de mercado no futebol profissional de forma orientada por dados e alinhada a critérios de eficiência esportiva e financeira.

2 OBJETIVOS

2.1 Objetivo Geral

O presente estudo tem como objetivo principal desenvolver e validar um modelo de AM capaz de prever o desempenho de equipes de futebol a partir de dados estatísticos de desempenho individual de jogadores. Neste trabalho, entende-se por desempenho a pontuação esperada de uma equipe ao longo de uma temporada de uma competição esportiva.

Entretanto, o objetivo central não se limita ao desenvolvimento do modelo preditivo em si. Pretende-se que tal modelo seja utilizado como componente de algoritmos de otimização voltados ao apoio à tomada de decisão na montagem de elencos. Em particular, seja f o modelo de AM obtido e seja X um arranjo contendo as *features* correspondentes ao elenco de um determinado time. Embora o modelo possa ser utilizado para prever o desempenho de equipes existentes, uma vez treinado ele também pode ser empregado para estimar o desempenho de equipes hipotéticas.

Dessa forma, considere P o conjunto de todos os elencos X' que podem ser obtidos a partir do elenco atual X por meio da contratação de novos jogadores, respeitando um determinado orçamento disponível. O objetivo passa então a ser identificar, dentre todas as combinações possíveis de jogadores em P, aquela que maximiza o desempenho previsto pelo modelo de AM. Formalmente, busca-se resolver o problema

$$\max_{X' \in P} f(X').$$

Observa-se, portanto, que o problema estudado configura um problema de otimização combinatória, no qual o modelo de AM desempenha o papel de função objetivo.

2.2 Objetivos Específicos

Com o intuito de atingir o objetivo geral, foram estabelecidos os seguintes objetivos específicos:

1. Coletar, estruturar e processar dados históricos de jogadores e partidas, abrangendo múltiplas temporadas, garantindo a qualidade e a integridade das informações utilizadas no desenvolvimento dos modelos preditivos.

2. Desenvolver uma metodologia para quantificar o impacto individual dos jogadores, combinando estatísticas históricas ao longo das temporadas por meio de um sistema de ponderação temporal, priorizando desempenhos mais recentes.

3. Construir modelos de AM para prever o desempenho agregado de uma equipe em uma determinada temporada, considerando a composição do elenco e os dados históricos dos jogadores.

4. Validar a eficácia dos modelos preditivos, comparando as estimativas com os resultados reais de temporadas subsequentes, utilizando métricas estatísticas como erro médio absoluto (MAE) e coeficiente de determinação (R^2).

5. Desenvolver modelos simples de otimização baseados no modelo de AM selecionado, visando a identificação de combinações de jogadores que maximizem o desempenho esperado da equipe sob restrições de mercado.

Ao integrar técnicas de AM e otimização, este estudo busca fornecer uma ferramenta analítica de apoio à decisão para a montagem de elencos no futebol profissional, contribuindo para práticas de gestão esportiva orientadas por dados.

3 REFERENCIAL TEÓRICO

3.1 Aprendizado de Máquina

Aprendizado de Máquina (AM) é o estudo de algoritmos e modelos estatísticos empregados por sistemas computacionais para realizar tarefas específicas de forma autônoma. Essa abordagem permite que as máquinas aprendam a partir de dados e melhorem seu desempenho sem necessidade de reprogramação explícita (MAHESH, 2020). Segundo Geron (2019), AM pode ser descrito como a ciência (ou arte) de programar computadores para que aprendam com os dados.

Um exemplo clássico de aplicação de AM é o filtro de *spam*, que permite que o sistema identifique automaticamente emails indesejados, diferenciando-os de mensagens legítimas com base em padrões aprendidos (GéRON, 2019).

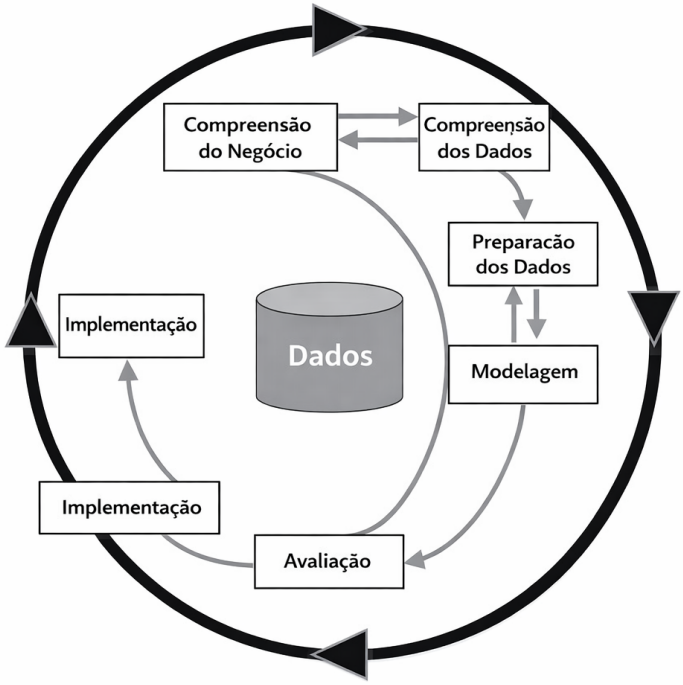
Com a crescente disponibilidade de *datasets*, o uso de AM tem apresentado um notável crescimento (MAHESH, 2020). Esse avanço pode ser atribuído à maior disponibilidade de dados e ao aumento da capacidade computacional. A constante conexão das pessoas a dispositivos digitais gera volumes massivos de dados, enquanto tecnologias modernas permitem o processamento eficiente dessas grandes quantidades de informação (JORDAN; MITCHELL, 2015).

Muitas indústrias têm adotado o AM com o objetivo de extrair insights relevantes de *databases*, facilitando o aprendizado a partir dos dados (MAHESH, 2020). Além disso, o AM tem sido amplamente aplicado em diversas áreas, incluindo reconhecimento facial, segmentação de imagens, detecção de câncer e previsão de classes e resultados, destacando-se também no contexto esportivo (ALMULLA et al., 2023).

No desenvolvimento de sistemas de AM, a construção de modelos geralmente ocorre dentro de um pipeline composto por múltiplas etapas de processamento de dados. Em um projeto típico de AM, os dados passam inicialmente por fases de coleta e preparação, seguidas por etapas de exploração e preparação

dos dados, engenharia de atributos, seleção e treinamento de modelos, além de procedimentos de avaliação e ajuste de hiperparâmetros (GéRON, 2019). Essas etapas são frequentemente organizadas em forma de pipeline, no qual cada componente executa uma transformação específica sobre os dados e produz uma saída que será utilizada pela etapa subsequente do processo (GéRON, 2019). Essa estrutura modular permite organizar o fluxo de processamento e facilita a reprodução e manutenção dos modelos desenvolvidos. Um exemplo de organização dessas etapas no ciclo de vida de um projeto de análise de dados é apresentado na Figura **??**, baseada no processo CRISP-DM, amplamente utilizado em projetos de mineração de dados e AM (KELLEHER; NAMEE; D'ARCY, 2015).

Figura 3.1 – Pipeline CRISP-DM



Fonte: Adaptado de (KELLEHER; NAMEE; D'ARCY, 2015).

Dentro desse pipeline, a preparação e transformação dos dados desempenham papel fundamental, uma vez que os algoritmos de aprendizado frequentemente exigem representações específicas dos atributos para funcionar adequadamente. Em particular, alguns métodos operam apenas com atributos nominais ou podem se beneficiar de representações discretizadas dos dados. Nesse contexto, atributos numéricos podem ser transformados em intervalos discretos antes da etapa de aprendizado, processo conhecido como discretização (WITTEN et al., 2016).

Diferentes abordagens podem ser empregadas para realizar essa transformação. Métodos não supervisionados podem dividir o domínio dos atributos em intervalos pré-definidos, como na discretização por intervalos de mesma largura ou mesma frequência. Já métodos supervisionados utilizam informações das classes presentes nos dados para determinar os pontos de divisão mais apropriados, frequentemente empregando critérios baseados em entropia e ganho de informação (WITTEN et al., 2016). Dessa forma, a transformação dos dados constitui uma etapa importante do pipeline de aprendizado de máquina, pois permite adaptar a representação dos atributos às características dos algoritmos utilizados e pode influenciar diretamente a qualidade do modelo gerado. AM utiliza diferentes algoritmos para resolver problemas relacionados com dados. Cientistas de dados destacam que não há um único algoritmo universalmente superior para resolver todos os tipos de problemas (MAHESH, 2020). Esta seção apresenta uma breve descrição de alguns dos modelos mais comumente utilizados em AM para a resolução de problemas.

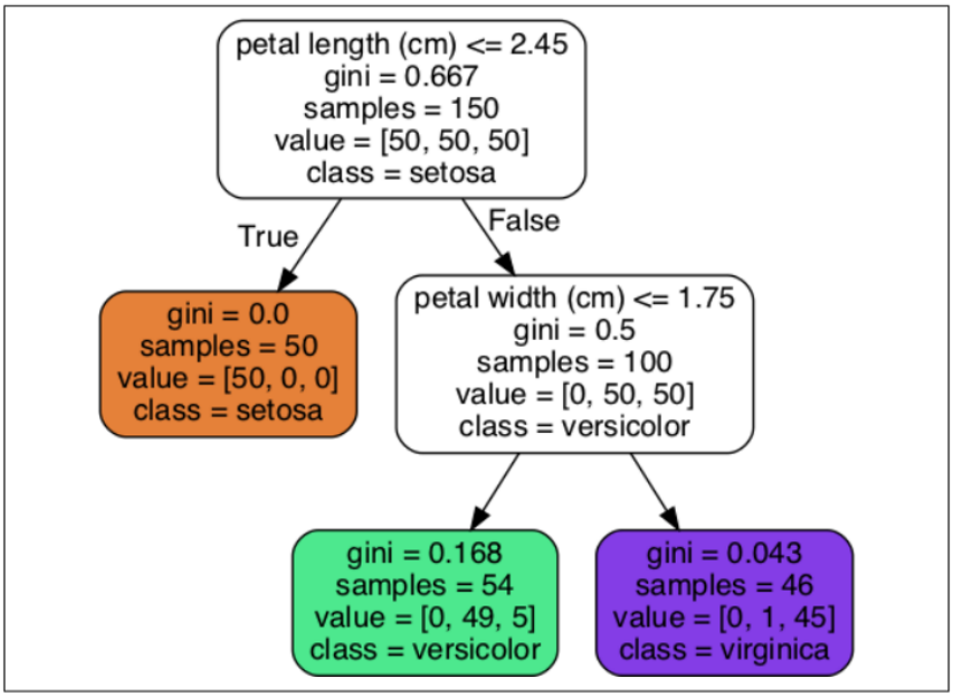
3.1.1 Árvore de Decisão (AD)

As Árvores de Decisão são algoritmos versáteis de AM que podem ser utilizados tanto para tarefas de classificação, quanto para tarefas de regressão (GÉRON, 2019). Esses algoritmos recebem um *input* de dados de treinamento (con-

junto de atributos independentes + atributo classe) e retornam como saída uma árvore de decisão que permite classificar uma nova instância a partir de seus atributos independentes (GéRON, 2019).

Um exemplo de AD pode ser visto na Figura 3.2

Figura 3.2 – AD para Iris



Fonte: Geron, 2019

Na Figura 3.2, é possível observar uma árvore de decisão utilizada para classificar flores do conjunto de dados *Iris*. O nó raiz divide as amostras com base no comprimento da pétala (*petal length* $\leq$ 2.45), separando a classe *setosa* (pura, Gini = 0). As amostras restantes são subdivididas pelo critério de largura da pétala (*petal width* $\leq$ 1.75), onde as classes *versicolor* e *virginica* são diferenciadas com alta precisão (Gini próximo de 0 nos nós finais). A árvore otimiza a classificação minimizando o índice de Gini em cada divisão (GéRON, 2019).

3.1.2 Random Forest (RF)

O RF consiste em uma combinação de AD, geralmente treinadas por meio do método *bagging* (GéRON, 2019). Diferente das ADs, RF utiliza uma randomização extra no crescimento das árvores. Ao invés de buscar a melhor *feature* para dividir o nó, utiliza-se da melhor dentro de um *subset* randômico. Nesse caso, busca-se reduzir a variância do modelo por meio da agregação de múltiplas árvores, mantendo um viés relativamente baixo quando comparado a modelos excessivamente regularizados.

3.1.3 Support Vector Machine (SVM)

O *Support Vector Machine* (SVM) é um algoritmo de AM supervisionado utilizado para tarefas de classificação e regressão. O seu princípio básico é encontrar um hiperplano ótimo que separe os dados em diferentes classes, maximizando a margem entre os pontos mais próximos das classes opostas (NOBLE, 2006).

O hiperplano utilizado pelo SVM consiste em uma superfície de decisão que separa os pontos de dados em classes distintas. Em um espaço 2D, ele é uma linha; em 3D, um plano e, em dimensões superiores, uma hiper-superfície (NOBLE, 2006)

Cabe ressaltar que nem sempre os dados são linearmente separáveis. Sendo assim, o SVM utiliza funções *kernel* para projetar os dados em um espaço de maior dimensão, onde a separação linear se torna possível (NOBLE, 2006). As principais funções *kernel* são:

1. **Linear**: Quando os dados já são linearmente separáveis.
2. **Polinomial**: Para capturar relações não lineares.
3. **RBF (Radial Basis Function - Gaussiana)**: Amplamente usada para capturar padrões complexos.
4. **Sigmoid**: Similar a redes neurais.

3.1.4 Gradient Boosting (XGBoost)

O *Gradient Boosting* é uma técnica de aprendizado baseada na combinação sequencial de modelos fracos, geralmente árvores de decisão rasas, com o objetivo de minimizar uma função de perda diferenciável (GéRON, 2019). Diferentemente do *bagging*, no qual os modelos são treinados de forma independente, o *boosting* constrói cada novo preditor de maneira iterativa, ajustando-o aos resíduos (erros) cometidos pelo modelo anterior. Esse processo pode ser interpretado como uma otimização por gradiente no espaço funcional, na qual cada novo modelo aproxima a direção de maior redução da função de custo.

Formalmente, a cada iteração m, busca-se adicionar um novo modelo $h_m(x)$ de forma que o modelo final seja dado por:

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x),$$

onde γ_m representa o passo ótimo na direção do novo preditor.

O *XGBoost* (*Extreme Gradient Boosting*) constitui uma implementação otimizada do *Gradient Boosting*, incorporando técnicas de regularização, paralelização e controle eficiente de memória (GéRON, 2019). Além disso, sua função objetivo inclui termos de penalização que auxiliam na prevenção de *overfitting*, tornando-o especialmente eficaz em problemas de alta dimensionalidade e grandes volumes de dados.

3.1.5 Multilayer Perceptron (MLP)

O *Multilayer Perceptron* (MLP) é uma função matemática parametrizada que aprende a melhor forma de transformar um vetor de entrada $\mathbf{x}$ em uma saída $\mathbf{y}$. Assim, o MLP pode ser entendido como uma composição de várias funções aplicadas em sequência, formando um modelo de múltiplas camadas (GOODFELLOW; BENGIO; COURVILLE, 2016).

A arquitetura do MLP segue uma lógica de entrada $\rightarrow$ camadas ocultas $\rightarrow$ saída, utilizando uma função de ativação $g(\cdot)$ para introduzir não linearidade em cada camada. Essa relação é expressa por:

$$h^{(l)} = g\left(W^{(l)}h^{(l-1)} + b^{(l)}\right),$$

onde $W^{(l)}$ e $b^{(l)}$ representam, respectivamente, a matriz de pesos e o vetor de vieses da camada l (GOODFELLOW; BENGIO; COURVILLE, 2016).

A introdução da função não linear $g(\cdot)$ é essencial, pois permite que o MLP aprenda relações complexas entre as variáveis de entrada e saída. Sem ela, o modelo se reduziria a uma simples combinação linear, incapaz de representar funções não lineares (GOODFELLOW; BENGIO; COURVILLE, 2016).

3.1.6 Generalização, Viés e Variância

Um dos principais desafios em AM é garantir que o modelo apresente boa capacidade de generalização, isto é, desempenho satisfatório em dados não utilizados durante o treinamento (GOODFELLOW; BENGIO; COURVILLE, 2016).

Modelos excessivamente complexos podem se ajustar não apenas aos padrões relevantes, mas também ao ruído presente no conjunto de treinamento, fenômeno conhecido como *overfitting*. Em contrapartida, modelos demasiadamente simples podem não capturar relações importantes nos dados, resultando em *underfitting*.

Esse comportamento está diretamente relacionado ao equilíbrio entre viés e variância. Modelos com alto viés tendem a apresentar subajuste, enquanto modelos com alta variância tornam-se excessivamente sensíveis às particularidades do conjunto de treinamento. Técnicas como validação cruzada, regularização e controle da complexidade do modelo são estratégias amplamente utilizadas para mitigar esses problemas (GéRON, 2019).

3.1.7 Seleção de Atributos e Redução de Dimensionalidade

Em problemas com elevado número de variáveis explicativas, a presença de atributos redundantes ou irrelevantes pode impactar negativamente o desempenho dos modelos preditivos. A seleção de atributos busca identificar subconjuntos de variáveis que contribuam de forma significativa para a tarefa de aprendizagem (LIU et al., 2010).

Os métodos de seleção podem ser classificados em três categorias principais: filtros, que utilizam medidas estatísticas independentes do modelo; wrappers, que avaliam subconjuntos de atributos com base no desempenho de um algoritmo específico; e métodos embutidos, nos quais a seleção ocorre durante o próprio processo de treinamento (GUYON et al., 2006).

Além da seleção direta de atributos, técnicas de redução de dimensionalidade, como a Análise de Componentes Principais (PCA), realizam uma transformação linear das variáveis originais em um novo conjunto de componentes ortogonais que preservam a maior parte da variância dos dados. Essa abordagem pode reduzir complexidade computacional e mitigar problemas de multicolinearidade. Cabe ressaltar que a PCA não consiste em um método de seleção de atributos, mas sim em uma técnica de extração de atributos, pois cria novas variáveis a partir de combinações lineares das variáveis originais (GUYON et al., 2006).

As técnicas apresentadas nesta seção constituem a base teórica para os modelos preditivos empregados neste trabalho, sendo posteriormente integradas a métodos de otimização para apoio à tomada de decisão no contexto esportivo.

3.2 Pesquisa Operacional

Como tratado anteriormente, a área da PO consiste em um campo multidisciplinar que aplica modelos matemáticos, estatísticos e computacionais. PO vem sendo amplamente utilizada em diversas áreas organizacionais, tais como manu-

fatura, transportes, construção civil, telecomunicações, planejamento financeiro, planos de saúde, serviço público, entre outros (HILLIER; LIEBERMAN, 2001).

Conforme discutido na seção anterior, problemas esportivos como montagem de elenco e planejamento competitivo podem ser formalizados como problemas de otimização matemática sob restrições estruturais e orçamentárias. Dependendo da natureza do problema, tais formulações podem recorrer a abordagens heurísticas voltadas à resolução de problemas de otimização combinatória, nos quais o espaço de soluções possíveis cresce exponencialmente com o número de variáveis de decisão.

Nos tópicos seguintes, são detalhados os principais métodos de otimização utilizados na modelagem de problemas combinatórios, com destaque para abordagens heurísticas que fundamentam a metodologia proposta neste trabalho.

3.2.1 Algoritmo Guloso

Em problemas de otimização, um algoritmo é dito guloso quando o mesmo faz uma escolha localmente ótima, na esperança de que essa escolha leve a uma solução globalmente ótima (CORMEN et al., 2009). Diferentemente de abordagens exaustivas, algoritmos gulosos tomam decisões irrevogáveis a cada etapa, baseando-se apenas na informação disponível no momento da escolha. É importante destacar que, conforme ressaltado por (CORMEN et al., 2009), algoritmos gulosos não garantem, em geral, a obtenção de soluções globalmente ótimas, sendo necessários argumentos específicos de corretude para justificar quando a estratégia gulosa é válida.

Conforme discutido por (CORMEN et al., 2009), algoritmos gulosos podem ser formulados de maneira iterativa ou recursiva. Na forma recursiva, o método costuma seguir um padrão *top-down*: realiza-se uma escolha gulosa, reduz-se o problema a um subproblema estritamente menor e repete-se o processo até a construção completa de uma solução. A justificativa de corretude, quando pos-

sível, normalmente se apoia em dois ingredientes: (i) a *propriedade da escolha gulosa*, isto é, a existência de uma solução ótima que contém a decisão local tomada; e (ii) a *subestrutura ótima*, segundo a qual, após fixar a decisão gulosa, o restante do problema preserva a mesma estrutura do original, de modo que uma solução ótima do subproblema completa uma solução ótima do problema inicial.

Para ilustrar o funcionamento dessa estratégia, considere o problema do *Traveling Salesman Problem* (TSP), no qual se deseja encontrar um circuito Hamiltoniano de custo mínimo que visite exatamente uma vez cada cidade de um conjunto dado e retorne ao ponto inicial. Nesse contexto, uma heurística gulosa construtiva bastante conhecida é o método do *vizinho mais próximo* (*nearest neighbor*). O algoritmo inicia em uma cidade arbitrária e, a cada etapa, seleciona a cidade ainda não visitada que possui a menor distância em relação à cidade atual, repetindo o processo até que todas as cidades tenham sido visitadas e o ciclo seja fechado retornando à cidade inicial (GENDREAU; POTVIN, 2010).

Esse procedimento exemplifica claramente a lógica gulosa: em cada passo é realizada a escolha localmente mais vantajosa — visitar a cidade mais próxima — sem reconsiderar decisões anteriores. Embora tal estratégia permita construir rapidamente uma solução viável para o problema, não há garantia de que o circuito obtido seja globalmente ótimo, uma vez que decisões tomadas nas primeiras etapas podem restringir significativamente as escolhas disponíveis nas etapas subsequentes (**??**).

O Algoritmo **??** apresenta o pseudocódigo dessa heurística gulosa aplicada ao TSP.

No contexto deste trabalho, um algoritmo guloso é utilizado de forma recursiva com o objetivo de gerar uma solução inicial para o problema de otimização da montagem de elencos. Dado um time e um orçamento, a cada iteração, o jogador com menor minutagem do elenco é removido e substituído por candidatos externos. Para cada substituição possível, o modelo de AM é empregado como

Algorithm 1 Algoritmo Guloso com estratégia First Improvement

```
Obter lista de jogadores candidatos da temporada anterior
Obter elenco atual do time alvo
Ordenar jogadores do elenco atual por menor minutagem
S ← ∅                                                    ▷ novas contratações
calcular pontuação atual
for i = 1 até size do
    for cada jogador candidato p do
        if p não pertence ao elenco atual e cabe no orçamento then
            Simular substituição do jogador de menor minutagem por p
            Calcular nova pontuação prevista
            if pontuação melhora then
                adicionar p em S
                atualizar orçamento
                atualizar pontuação
                break                                    ▷ first improvement
            end if
        end if
    end for
    if nenhuma melhora encontrada then
        break
    end if
end for
return conjunto de contratações e pontuação prevista
```

função objetivo, estimando a pontuação esperada do time resultante. A escolha gulosa consiste, portanto, em selecionar o(s) jogador(es) que cabem no orçamento do time e cuja inclusão maximiza a pontuação prevista, mantendo-se fixas as demais variáveis do elenco. Assim como no exemplo discutido anteriormente, a solução é construída incrementalmente por decisões locais irrevogáveis, produzindo rapidamente uma solução factível que servirá como ponto de partida para os procedimentos de refinamento descritos na próxima seção. Nesse processo, o modelo de AM atua como função objetivo, avaliando o desempenho esperado das diferentes composições de elenco geradas pelo algoritmo de otimização.

3.2.2 Algoritmo de Busca Local

A busca local é uma técnica heurística amplamente utilizada em problemas de otimização combinatória, cujo objetivo é aprimorar uma solução inicial factível por meio da exploração iterativa de sua vizinhança (GENDREAU; POTVIN, 2010). Diferentemente de métodos exatos, a busca local não realiza uma exploração global do espaço de soluções, concentrando-se na melhoria incremental da solução corrente a partir de movimentos locais.

Conforme descrito por (GENDREAU; POTVIN, 2010), um algoritmo de busca local é composto essencialmente por quatro elementos: uma solução inicial, a definição de uma vizinhança, um critério de aceitação de movimentos e uma condição de parada. A vizinhança determina o conjunto de soluções que podem ser alcançadas a partir da solução atual por meio de pequenas modificações estruturais, enquanto o critério de aceitação estabelece se uma solução vizinha deve ou não substituir a solução corrente. O processo é repetido iterativamente até que nenhuma melhoria adicional seja encontrada ou que um critério de parada seja satisfeito.

Em contraste com algoritmos gulosos, métodos de busca local partem de uma solução inicial e buscam iterativamente soluções vizinhas que melhorem o

valor da função objetivo. O processo continua enquanto existirem melhorias locais, terminando quando se atinge um ponto no qual nenhuma modificação local resulta em melhora adicional.

Um exemplo clássico apresentado por (CORMEN et al., 2009) surge no problema de agrupamento (*k-means clustering*). Dado um conjunto de pontos e um número fixo de centros k, deseja-se encontrar uma configuração de centros que minimize a soma das distâncias quadráticas entre cada ponto e o centro mais próximo. Como esse problema é NP-difícil, não se conhece um algoritmo polinomial que garanta a obtenção do ótimo global.

Nesse contexto, Lloyd propôs um procedimento iterativo simples que encontra um mínimo local da função objetivo. O método inicia com uma escolha inicial de centros e repete duas etapas: (i) atribuir cada ponto ao centro mais próximo; e (ii) recalcular cada centro como a média dos pontos atribuídos ao respectivo cluster. O processo é repetido até que nenhuma alteração reduza o valor da função objetivo, caracterizando a convergência para um mínimo local.

Esse procedimento ilustra claramente o princípio da busca local: a partir de uma solução inicial, o algoritmo explora soluções vizinhas obtidas por pequenas modificações estruturais, aceitando apenas mudanças que melhoram a função objetivo.

No problema abordado neste trabalho, a busca local é empregada para avaliar diferentes conjuntos de jogadores candidatos a contratação, respeitando uma restrição de orçamento. Cada solução é representada por um conjunto de jogadores de cardinalidade fixa, sendo a vizinhança definida por operações de substituição unitária (*swap neighborhood*). Em cada iteração, os jogadores do conjunto atual com menor minutagem são selecionados como candidatos à remoção e substituídos por jogadores externos ainda não selecionados, produzindo soluções vizinhas que preservam o tamanho do elenco. Cada solução vizinha gerada é avaliada por

meio de uma função objetivo, implementada como um modelo de AM, responsável por estimar a pontuação esperada do time na temporada considerada.

A restrição orçamentária é tratada como condição de factibilidade: soluções que excedem o orçamento estabelecido são descartadas durante o processo de busca. O critério de aceitação adotado é de melhoria estrita, isto é, uma solução vizinha é aceita apenas se apresentar valor superior da função objetivo em relação à solução corrente. O algoritmo é encerrado quando nenhuma substituição viável resulta em melhoria adicional, caracterizando a convergência para um ótimo local, conforme descrito em (GENDREAU; POTVIN, 2010).

4 REVISÃO DE LITERATURA

4.1 Aprendizado de Máquina no contexto esportivo

Em 2020, *Horvat & Job* apresentaram uma revisão abrangente sobre o uso de AM na predição de resultados esportivos, com foco principal em esportes coletivos, especialmente o futebol. Os autores destacam que o AM é particularmente adequado a essa tarefa, uma vez que modalidades esportivas geram grandes volumes de dados estruturados e não estruturados, provenientes de eventos de jogo, sistemas de *tracking*, probabilidades associadas às partidas e bases históricas consolidadas (HORVAT; JOB, 2020).

Conforme apontado por *Horvat & Job* (2020), a maior parte dos estudos que empregam métodos preditivos no contexto esportivo concentra-se na predição de resultados de partidas individuais, especialmente na classificação de vitórias, empates e derrotas. Essa predominância pode ser explicada tanto pelo interesse mercadológico associado às apostas esportivas, que incentiva o desenvolvimento de modelos voltados à antecipação de resultados (ABIDIN, 2021), quanto pela própria natureza técnica do problema, que se ajusta adequadamente aos algoritmos supervisionados mais difundidos e à disponibilidade de bases históricas amplamente registradas no futebol (HORVAT; JOB, 2020). Nesse contexto, observa-se menor atenção a problemas relacionados ao desempenho agregado ao longo de uma temporada, especialmente quando este é modelado de forma prospectiva como função da composição individual do elenco.

Em contraste com essa predominância de estudos voltados à predição de partidas isoladas, a revisão conduzida por (ABIDIN, 2021) identifica, ainda que em menor número, trabalhos que abordam diretamente o problema de seleção e formação de equipes sob uma perspectiva multicritério e de otimização. Entre os estudos mencionados, destacam-se (TAVANA et al., 2013), que propõem um sistema de inferência fuzzy em duas fases para seleção de jogadores e posterior

arranjo tático da equipe, e (QADER et al., 2017), que estruturam o problema de escolha de atletas como uma análise multicritério baseada em múltiplas métricas de desempenho. Tais abordagens representam esforços relevantes no sentido de formalizar o processo de composição de elencos, ainda que se apoiem predominantemente em modelos fuzzy ou técnicas de agregação determinística, sem incorporar mecanismos de aprendizado supervisionado voltados à predição prospectiva de desempenho coletivo.

Os estudos voltados à predição de resultados, por sua vez, identificam o uso recorrente de algoritmos como regressão logística, *Support Vector Machines* (SVM), *Random Forest* e redes neurais, destacando que modelos que integram múltiplas fontes de informação — incluindo estatísticas tradicionais, indicadores derivados e variáveis contextuais — tendem a apresentar melhor desempenho preditivo (HORVAT; JOB, 2020). Ainda assim, a tarefa permanece desafiadora devido à natureza estocástica do futebol e à ausência de padronização entre bases de dados utilizadas em diferentes estudos. Cabe ressaltar que, nos últimos anos, a geração de dados aumentou significativamente com a maior disponibilidade de dados esportivos em larga escala, impulsionada pela expansão de sistemas de *tracking* e por bancos estruturados como *Opta* e *StatsBomb* (HORVAT; JOB, 2020).

4.1.1 Previsão de resultados de partidas

Conforme ressaltado anteriormente, os modelos de previsão de resultados de partidas constituem uma das aplicações mais tradicionais do AM no futebol, com foco na classificação de vitórias, empates e derrotas. A seguir, apresentam-se alguns estudos representativos dessa linha de pesquisa, com o objetivo de evidenciar sua evolução metodológica.

Um dos trabalhos pioneiros é o de *Joseph, Fenton & Neil* (2006) (JOSEPH; FENTON; NEIL, 2006), que empregaram redes Bayesianas para prever resultados a partir de um conjunto reduzido de dados do *Tottenham Hotspur Foot-*

ball Club (1995–1997). A base analisada compreendeu 76 partidas e 30 atributos, sendo 28 jogadores representados por variáveis binárias (presença ou ausência), além do local da partida e da qualidade do adversário. Os próprios autores reconhecem tratar-se de um cenário com disponibilidade limitada de dados, o que impõe desafios adicionais aos métodos de aprendizado supervisionado. Ainda assim, os resultados indicaram que a rede Bayesiana construída com conhecimento especializado apresentou melhor desempenho relativo nos experimentos realizados, mesmo diante da limitação amostral. Esse resultado reforça que, em contextos com bases reduzidas e forte especificidade estrutural, a modelagem explícita das relações entre variáveis pode mitigar parcialmente os efeitos da escassez de dados. Entretanto, o estudo permanece circunscrito à predição de partidas individuais e a uma equipe específica.

Em contexto mais recente, *Baboota & Kaur* (2019) (BABOOTA; KAUR, 2019) investigaram a predição multiclasse de resultados na Premier League utilizando dados de duas temporadas (2014–2015 e 2015–2016). O estudo incorporou variáveis relacionadas ao desempenho recente das equipes, estatísticas técnicas de jogo (finalizações, posse de bola, escanteios), *ratings* derivados do FIFA Index e fator casa/fora. Foram avaliados algoritmos como *Random Forest*, SVM, Naive Bayes e XGBoost, sendo o *Gradient Boosting* o modelo de melhor desempenho. Ainda assim, os autores observam que os modelos não superaram as previsões das casas de apostas, que incorporam variáveis não observáveis pelos modelos estatísticos. Embora represente uma ampliação substancial em relação a trabalhos anteriores — tanto em volume de dados quanto na diversidade de algoritmos avaliados — o problema permanece centrado na predição do desfecho de partidas individuais.

No âmbito do Aprendizado Profundo, *Awadallah & Khandelwal* (2020) (AWADALLAH; KHANDELWAL, 2020) compararam regressão logística, *Random Forest*, redes neurais densas (DNN) e LSTM na predição de resultados da

EPL ao longo de dez temporadas (2008–2019), utilizando aproximadamente 4.180 partidas e um conjunto reduzido de 24 *features*. O modelo LSTM apresentou o melhor desempenho, alcançando cerca de 58% de acurácia em teste. Apesar do avanço metodológico representado pelo uso de redes neurais profundas, os autores destacam limitações relacionadas à generalização e à natureza estocástica do futebol.

De modo geral, observa-se uma evolução metodológica significativa, desde bases reduzidas com forte dependência de conhecimento especializado até modelos supervisionados e redes neurais aplicados a bases mais amplas. Ainda assim, a unidade analítica predominante permanece sendo a partida individual, com ênfase na classificação do resultado do jogo.

4.1.2 Aplicações recentes do AM no futebol

Para além da predição de partidas isoladas, a literatura recente evidencia uma ampliação do escopo de aplicação do AM no futebol. *Rico-González et al.* (2023) demonstram que o AM tem sido empregado em diversas frentes, como análise de desempenho físico, detecção de padrões táticos, avaliação de risco de lesões e modelagem espacial com dados de *tracking* (RICO-GONZÁLEZ et al., 2023). A crescente disponibilidade de dados tem impulsionado abordagens metodológicas cada vez mais complexas, incluindo redes neurais profundas, modelos probabilísticos e métodos baseados em séries temporais.

Além das aplicações voltadas à predição de partidas, estudos recentes evidenciam a incorporação crescente de dados espaciais e fisiológicos na modelagem do desempenho esportivo. O volume *Machine Learning and Data Mining for Sports Analytics* (MLSA, 2023) destaca a integração entre dados de eventos e informações de *tracking*, ressaltando que modelos exclusivamente baseados em eventos enfrentam limitações estruturais pela ausência de contexto espacial e pressão defensiva (BREFELD et al., 2024). De forma complementar, pesquisas como

a de Majumdar et al. (2022) exploram a aplicação de AM na predição de lesões a partir de variáveis relacionadas à carga de treino e competição, utilizando desde regressão logística até redes neurais artificiais. Essas abordagens evidenciam a ampliação temática e metodológica do campo, mas permanecem majoritariamente voltadas à modelagem de situações específicas de jogo ou risco individual, não se direcionando à modelagem prospectiva do desempenho agregado de temporada como função da composição do elenco.

No contexto nacional, observa-se também o crescimento da área de *sports analytics*, com iniciativas acadêmicas como o Sports Analytics Lab (SALab) da UFMG, voltadas à aplicação de métodos quantitativos no futebol.

Apesar dessa expansão metodológica e temática, observa-se que a maior parte dos estudos ainda se concentra na modelagem de eventos específicos ou na predição de partidas isoladas, conforme destacado por *Horvat & Job* (2020). Em contraste, verifica-se menor atenção a abordagens voltadas à modelagem do desempenho agregado das equipes ao longo de uma temporada, especialmente quando esse desempenho é tratado como função da composição individual do elenco. Essa constatação reforça a necessidade de examinar de forma mais sistemática como a literatura tem abordado o desempenho agregado sob perspectivas estatísticas e econômicas.

4.1.3 Desempenho agregado e determinantes de temporada

Além da predição de partidas individuais, parte da literatura tem investigado determinantes do desempenho agregado ao longo de uma temporada. Lago-Ballesteros e Lago-Peñas (2010), ao analisarem todas as 380 partidas da temporada 2008–2009 da La Liga espanhola, identificaram diferenças estatisticamente significativas entre equipes do topo e da parte inferior da tabela em variáveis como gols marcados, total de finalizações, finalizações no alvo, assistências e posse de bola (LAGO-BALLESTEROS; LAGO-PEñAS, 2010). Os resultados indicam que

o sucesso competitivo agregado pode ser parcialmente explicado por padrões estatísticos observáveis ao longo da temporada.

Sob a ótica econômica, estudos como os de Carmichael, McHale e Thomas (2011) analisam o desempenho agregado das equipes a partir de variáveis estruturais, como gastos salariais e receitas, evidenciando associação entre investimento financeiro e posição final na liga (CARMICHAEL; MCHALE; THOMAS, 2011). De forma complementar, Totty e Owens (2011) discutem como mecanismos institucionais, como tetos salariais, influenciam o equilíbrio competitivo das ligas profissionais (TOTTY; OWENS, 2011). Essas abordagens indicam que o desempenho ao longo de uma temporada pode ser parcialmente explicado por fatores estruturais e de alocação de recursos, reforçando a ideia de que o sucesso competitivo agregado não é puramente aleatório.

Herold, Memmert e Perl (2019) investigaram a aplicação de AM no futebol profissional masculino, utilizando variáveis como passes, finalizações, dribles e métricas derivadas, como Gols Esperados (xG). O estudo demonstra que padrões técnicos individuais podem ser representados matematicamente e modelados por meio de algoritmos supervisionados e não supervisionados, incluindo redes neurais, árvores de decisão e regressão logística (HEROLD; MEMMERT; PERL, 2019). Tais evidências reforçam que estatísticas de desempenho individual constituem insumos mensuráveis e potencialmente preditivos do sucesso competitivo.

Complementarmente, Kite e Nevill (2017) investigaram preditores de sucesso ao longo de temporadas consecutivas de uma mesma equipe, demonstrando que determinados indicadores de desempenho apresentam capacidade explicativa sobre a posição final e os resultados obtidos (KITE; NEVILL, 2017). Tais resultados evidenciam que métricas de jogo podem ser utilizadas para modelar o sucesso competitivo.

Embora o estudo de *Kite e Nevill* (2017) represente um avanço ao empregar modelos estatísticos com finalidade preditiva, sua modelagem permanece centrada no nível da partida individual. O sucesso é operacionalizado a partir do desfecho dos jogos (vitória, empate ou derrota), e as variáveis explicativas derivam predominantemente de indicadores agregados por partida (KITE; NEVILL, 2017). Dessa forma, ainda que tais contribuições ampliem a compreensão dos determinantes estatísticos do desempenho competitivo, não há uma modelagem explícita do desempenho agregado da temporada como função da composição estrutural do elenco. Essa lacuna torna-se particularmente relevante quando se considera que decisões estratégicas no futebol profissional — especialmente no mercado de transferências — são tomadas com base na expectativa de desempenho ao longo de toda a competição, e não apenas em resultados isolados.

De modo semelhante, embora essas contribuições identifiquem determinantes técnicos e econômicos do sucesso competitivo, elas concentram-se majoritariamente na análise de resultados já observados ou em relações estruturais agregadas. Não modelam explicitamente, de forma prospectiva, a pontuação esperada de uma equipe como função direta da composição individual do elenco, tampouco integram tais estimativas a problemas formais de decisão.

4.2 Pesquisa Operacional no contexto esportivo

A Pesquisa Operacional (PO) constitui um conjunto de métodos analíticos voltados à modelagem e otimização de processos decisórios (MEDRI; YOTSUMOTO, 2009; WRIGHT, 2009). No contexto esportivo, sua aplicação abrange desde problemas estratégicos, como o gerenciamento de equipes, até decisões táticas e logísticas (GERCHAK, 1994).

Técnicas como Programação Linear, Programação Dinâmica e Processos de Markov têm sido empregadas na análise de substituições, estratégias de jogo e planejamento competitivo (CLARKE; NORMAN, 1998; HIROTSU; WRIGHT,

2002). Em muitos casos, tais problemas apresentam natureza combinatória e elevada complexidade computacional.

Conforme discutido por (KENDALL; PARKES; SPOERER, 2010), diversos problemas de organização competitiva e escalonamento esportivo (*sports scheduling*) apresentam elevada complexidade combinatória, sendo muitos deles classificados como NP-difíceis ou NP-completos em suas versões formais. Entre os exemplos destacados estão o Traveling Tournament Problem (TTP), que busca minimizar as distâncias percorridas pelas equipes sob restrições estruturais de calendário; o problema de minimização de *breaks* em torneios round-robin, cuja versão com calendário parcial é NP-difícil; e variantes do problema de alocação de árbitros, cuja versão de decisão é NP-completa. Diante dessa complexidade, especialmente em instâncias de grande porte e com múltiplas restrições, a literatura recorre amplamente a heurísticas e metaheurísticas, como busca local, *simulated annealing*, *tabu search* e métodos híbridos, visando à obtenção de soluções de alta qualidade em tempo computacional viável.

Além da dificuldade computacional inerente a esses problemas, observa-se também uma limitação recorrente na modelagem adotada. Na maior parte dos trabalhos, a função objetivo é definida por métricas simplificadas ou parâmetros fixos, não incorporando modelos preditivos baseados no desempenho individual de atletas como componente central do problema decisório. Assim, permanece inexplorada a integração entre modelos preditivos de desempenho agregado e técnicas de otimização aplicadas à montagem de elenco.

Problemas de montagem de elenco também têm sido abordados sob a ótica da otimização matemática. Pantuso (PANTUSO, 2017) propõe o *Football Team Composition Problem* por meio de um modelo de programação estocástica multietapa, no qual a incerteza sobre o desenvolvimento futuro dos atletas é incorporada explicitamente à formulação. De forma complementar, Boon e Sierksma (BOON; SIERKSMA, 2003) modelam a formação de equipes como um problema de alo-

cação ótima jogador–posição, formulado via programação linear inteira e equivalente a um problema de *matching* máximo ponderado. Já Becker e Sun (BECKER; SUN, 2015) aplicam uma estrutura análoga ao problema da mochila para otimizar decisões de seleção em *fantasy football* sob restrições orçamentárias.

Embora esses trabalhos avancem na formalização matemática da montagem de equipes, observa-se que a função objetivo permanece baseada em avaliações previamente definidas, pontuações atribuídas a priori ou estimativas paramétricas fixadas antes do processo de otimização. Não há, nesses estudos, a incorporação explícita de modelos preditivos de desempenho agregado aprendidos diretamente a partir de dados históricos como componente central da função objetivo. Dessa forma, permanece uma lacuna na literatura quanto à integração estruturada entre modelos de AM e problemas combinatórios de montagem de elenco.

4.2.1 Otimização Prescritiva e Problemas Combinatórios no Esporte

A transição de *predictive analytics* para *prescriptive analytics*, conforme discutido por (BERTSIMAS; KALLUS, 2019), consiste na integração entre modelos preditivos e problemas de otimização, permitindo que estimativas estatísticas sejam incorporadas diretamente na definição de políticas ótimas de decisão. Nesse paradigma, modelos de AM deixam de atuar apenas como ferramentas de previsão e passam a compor a função objetivo de problemas decisórios.

Além das abordagens clássicas de otimização, a literatura recente evidencia uma aproximação crescente entre PO e métodos de Inteligência Artificial (IA). Conforme discutido por (CHMAIT; WESTERBEEK, 2021), técnicas como aprendizado por reforço e algoritmos evolutivos vêm sendo empregadas em problemas decisórios no esporte, aproximando modelos de aprendizado e processos de otimização. Esse movimento reforça a convergência entre predição estatística e decisão prescritiva, abrindo espaço para estruturas integradas nas quais modelos de AM

podem ser incorporados diretamente à função objetivo de problemas combinatórios.

No contexto esportivo, decisões estratégicas como montagem de elenco e planejamento competitivo podem ser formalizadas como problemas de otimização sob restrições orçamentárias e estruturais. Tais problemas frequentemente assumem estrutura combinatória análoga àquela observada em problemas clássicos de escalonamento esportivo (KENDALL; PARKES; SPOERER, 2010), o que justifica o uso de heurísticas e metaheurísticas para obtenção de soluções viáveis.

O presente estudo insere-se explicitamente nessa interseção entre AM e PO, ao propor um modelo capaz de estimar a pontuação esperada de uma equipe em uma temporada a partir do desempenho individual de seus jogadores na temporada anterior. Diferentemente das abordagens focadas na previsão de partidas isoladas ou na análise retrospectiva de determinantes agregados, o modelo desenvolvido busca capturar prospectivamente o impacto da composição do elenco sobre o desempenho competitivo. Posteriormente, essa estimativa é incorporada como função objetivo em um problema combinatório de montagem de elenco sob restrição orçamentária, aproximando predição estatística e decisão prescritiva no contexto do futebol profissional.

5 METODOLOGIA

5.1 Coleta dos Dados

Com o intuito de coletar os dados, foi desenvolvido um algoritmo de *web scraping*, utilizando a biblioteca *Beautiful Soup* da linguagem *Python*. Essa biblioteca é amplamente empregada para extração e manipulação de informações a partir de páginas HTML e XML, permitindo a coleta automatizada e estruturada de grandes volumes de dados provenientes da web (Beautiful Soup Documentation, 2024). O algoritmo implementado automatiza o acesso às páginas de estatísticas, realiza a extração das tabelas correspondentes a cada clube e temporada e armazena os dados de forma padronizada para posterior análise.

O site utilizado para a extração foi o FBref, uma ampla base de dados de estatísticas de futebol que disponibiliza dados detalhados de jogadores e equipes de diversas competições ao redor do mundo. O FBref se destaca por fornecer estatísticas avançadas, como gols esperados (xG), ações de pressão, passes progressivos e estatísticas defensivas, tornando-se uma fonte relevante para análises preditivas e modelagem estatística no contexto do futebol.

Com o objetivo de utilizar um volume de dados superior ao empregado por *Baboota & Kaur* (2018) e *Joseph, Fenton & Neil* (2006), bem como reduzir o risco de *overfitting*, foram coletados os dados de todos os jogadores da *Serie A* italiana e da *Premier League* que atuaram ao longo de cinco temporadas (2019-2020, 2020-2021, 2021-2022, 2022-2023 e 2023-2024), além das tabelas de resultados de todos os jogos disputados nesse mesmo período.

A utilização de duas ligas distintas neste estudo possui o papel metodológico deevitar a ocorrência de *data leakage* entre os conjuntos de treinamento, teste e validação dos modelos de AM. Para isso, os dados de uma liga são utilizados exclusivamente na etapa de treinamento, enquanto os dados da outra liga são reservados para teste e validação. Essa estratégia assegura que o modelo seja

avaliado em um contexto completamente distinto daquele empregado no aprendizado, eliminando dependências estruturais entre amostras e proporcionando uma avaliação mais realista de sua capacidade de generalização.

Os dados obtidos foram armazenados no formato CSV, organizados em arquivos distintos para cada clube e temporada. Dessa forma, cada arquivo contém as estatísticas individuais dos jogadores de um determinado time em uma temporada específica, facilitando a organização, o pré-processamento e as etapas subsequentes da análise. Ao todo, foram coletados dados de 2900 jogadores pertencentes a 30 equipes da Serie A italiana e de 2399 jogadores pertencentes a 28 equipes da EPL. Assim, o conjunto de dados utilizado neste estudo totaliza 5299 registros de jogadores, distribuídos entre 58 equipes, que serviram de base para o desenvolvimento dos modelos preditivos.

5.2 Pré-processamento dos Dados

5.2.1 Base de Dados dos Jogadores

As tabelas extraídas do site FBref (FBref, 2026) para todos os times e temporadas foram estruturadas de forma idêntica, contendo 47 atributos relacionados à idade, posição, minutos jogados, bem como estatísticas ofensivas (gols, assistências, passes decisivos, entre outras) e defensivas (bloqueios, desarmes, erros graves, entre outras) de cada jogador. Esses dados foram carregados em diferentes *dataframes*, um para cada liga considerada, e posteriormente organizados por jogador, incluindo informações de clube, temporada e respectivos indicadores de desempenho. Ressalta-se que um mesmo atleta pode aparecer em diferentes temporadas e clubes dentro da organização adotada.

A Tabela 5.1 apresenta a descrição completa das variáveis utilizadas no estudo.

Tabela 5.1 – Descrição das variáveis utilizadas no modelo

Variável	Descrição
Identificação e contexto	
Player	Nome do jogador.
Season	Temporada à qual os dados se referem.
Team	Clube pelo qual o jogador atuou na temporada.
Pos.	Posição principal do jogador em campo.
Idade	Idade do jogador durante a temporada.
Participação e minutagem	
MP	Número de partidas disputadas.
Inícios	Número de partidas iniciadas como titular.
Min.	Total de minutos jogados.
90s	Minutos jogados convertidos em equivalentes de partidas completas (90 minutos).
Produção ofensiva (totais)	
Gols	Número total de gols marcados.
Assis.	Número total de assistências realizadas.
G+A	Soma de gols e assistências.
G-PB	Gols marcados excluindo penalidades.
PB	Número de pênaltis convertidos.
PT	Número de pênaltis cobrados.
Disciplina	
CrtsA	Cartões amarelos recebidos.
CrtV	Cartões vermelhos recebidos.
Métricas avançadas (totais)	
xG	*Expected Goals* (gols esperados), métrica que estima a probabilidade de finalizações resultarem em gol.
npxG	*Expected Goals* excluindo penalidades.

Variável	Descrição
xAG	*Expected Assists* (assistências esperadas).
npxG+xAG	Soma de gols esperados sem pênaltis e assistências esperadas.
Progressão	
PrgC	Corridas progressivas com a bola.
PrgP	Passes progressivos realizados.
PrgR	Recebimentos progressivos de passe.
Produção ofensiva (por 90 minutos)[a]	
Gols.1	Gols por 90 minutos[a].
Assis..1	Assistências por 90 minutos[a].
G+A.1	Gols mais assistências por 90 minutos[a].
G-PB.1	Gols sem pênaltis por 90 minutos[a].
G+A-PB	Gols mais assistências excluindo pênaltis por 90 minutos[a].
xG.1	*Expected Goals* por 90 minutos[a].
xAG.1	*Expected Assists* por 90 minutos[a].
xG+xAG	Soma de xG e xAG por 90 minutos[a].
npxG.1	*Expected Goals* sem pênaltis por 90 minutos[a].
npxG+xAG.1	Soma de npxG e xAG por 90 minutos[a].
Ações defensivas	
Div	Desarmes (*tackles*) realizados.
TklW	Desarmes vencidos.
Terço Def	Desarmes realizados no terço defensivo do campo.
Terço Central	Desarmes realizados no terço central do campo.
Terço de Ataque	Desarmes realizados no terço ofensivo.
Div.1	Tentativas de desarme.
Tent	Tentativas totais de desarme contra adversários.
Tkl%	Percentual de sucesso nos desarmes.
Perdido	Número de vezes em que o jogador foi driblado.

Variável	Descrição
Bloqueios e interceptações	
Bloqueios	Número de ações de bloqueio (chutes ou passes bloqueados).
TC	Chutes bloqueados.
Passe	Passes bloqueados.
Crts	Interceptações realizadas.
Tkl+Int	Soma de desarmes e interceptações.
Def	Ações defensivas realizadas na área defensiva.
Erros	Erros que resultaram em finalizações adversárias.

[a] Variáveis normalizadas por 90 minutos (equivalente a uma partida completa).

Fonte: FBref (2026), adaptado pelo autor.

Após a etapa de organização, os dados estruturados foram armazenados em um repositório público na plataforma GitHub, com o objetivo de garantir reprodutibilidade, transparência metodológica e facilidade de acesso para futuras análises. O repositório utilizado neste estudo está disponível em <https://github.com/LunusMax/epl-seriea-data>.

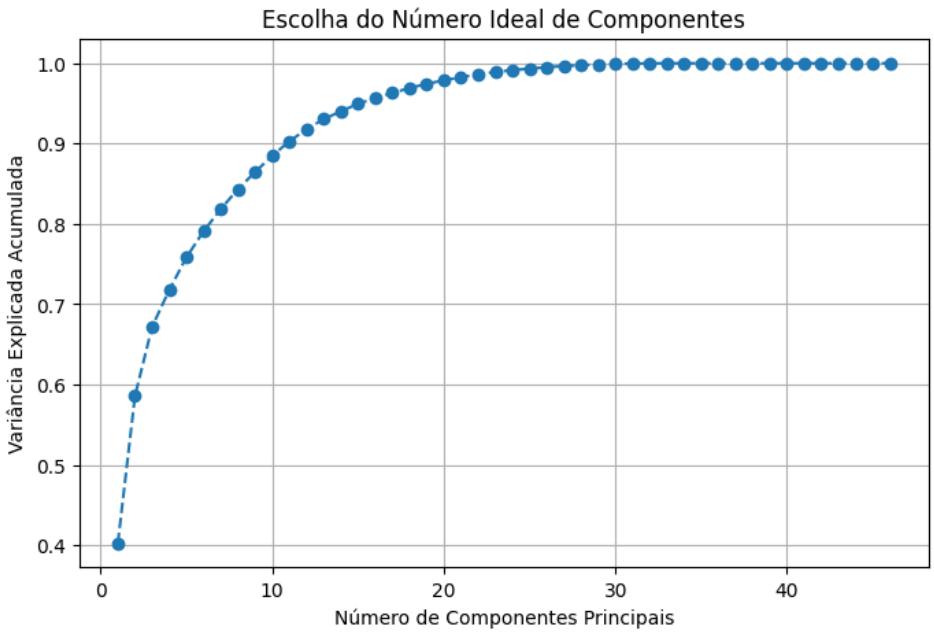
Em seguida, os *dataframes* correspondentes a cada liga foram concatenados em um único conjunto, englobando todos os jogadores considerados no estudo, o qual foi utilizado nas etapas subsequentes da análise.

Considerando o elevado número de atributos e a possível correlação entre variáveis, foi aplicada uma técnica de redução de dimensionalidade com o objetivo de mitigar *overfitting* e melhorar a eficiência computacional dos modelos de AM. Para isso, utilizou-se o Principal Component Analysis (PCA), conforme descrito na subseção a seguir.

5.2.1.1 Principal Component Analysis (PCA)

Para determinar o número ideal de componentes principais que a serem utilizados no PCA, foi gerado um *Scree Plot*, conforme ilustrado na Figura 5.1.

Figura 5.1 – Escolha do Número Ideal de Componentes do PCA



Fonte: do Autor, 2025

O gráfico apresenta a variância explicada acumulada em função do número de componentes principais, permitindo identificar o ponto de inflexão, ou seja, o momento em que a adição de novos componentes passa a contribuir de forma marginal para a variância total dos dados. O eixo x representa o número de componentes principais, enquanto o eixo y exibe a variância explicada acumulada.

A partir da Figura 5.1, observa-se que, acima de 20 componentes principais, o ganho de informação adicional se torna insignificante. Com base nessa análise, foram realizados testes com diferentes quantidades de componentes, garantindo que sempre eram utilizados no mínimo 20 componentes, de forma a preservar a maior quantidade possível de variância dos dados sem comprometer a

eficiência do modelo. Optou-se, por fim, pela utilização de 20 componentes principais, uma vez que esse valor já era suficiente para explicar 97,66% da variância total dos dados, indicando que praticamente toda a informação relevante do conjunto original estava preservada nessa representação reduzida.

5.2.2 Base de Dados dos Jogos

Os dados dos jogos realizados entre as temporadas 2019-2020 e 2023-2024, obtidos na etapa de coleta, estavam organizados em 14 colunas. No entanto, grande parte dessas informações era irrelevante para a análise (por exemplo, horário e data das partidas, público, local e árbitro). Assim, para otimizar o conjunto de dados, foram removidos os atributos considerados ruído, mantendo-se apenas três elementos essenciais: time mandante, time visitante e resultado da partida.

A coluna de resultados foi transformada em valores numéricos por meio de um algoritmo em Python, em processo análogo a um *encoder*: vitória do mandante foi codificada como '1', vitória do visitante como '2' e empate como '0'.

Com base nessa codificação, percorreu-se a base atribuindo três pontos a cada vitória e um ponto a cada empate. O resultado desse processamento foi uma tabela organizada por temporada, contendo cada clube e a respectiva pontuação acumulada.

5.3 Treinamento do Modelo de AM

Os dados de pontuação dos clubes foram organizados por temporada, enquanto as estatísticas individuais dos jogadores corresponderam sempre à temporada anterior. Dessa forma, o modelo utiliza informações conhecidas de desempenho prévio para estimar a pontuação final do clube na temporada subsequente. Adicionalmente, a pontuação dos clubes na temporada anterior foi incorporada como variável explicativa, de modo a capturar o histórico de desempenho recente da equipe.

O modelo de AM foi desenvolvido para prever a pontuação dos clubes ao longo das temporadas, com base no desempenho individual de seus jogadores. Para evitar vazamento de dados entre treinamento e teste, a base da Série A italiana foi utilizada no treinamento do modelo, enquanto a base da EPL foi destinada à etapa de teste.

5.3.1 Modelo de Aprendizado de Máquina

Para a etapa de previsão da pontuação dos clubes, foi implementado o modelo de MLP utilizando a biblioteca *Keras*. O modelo foi construído no formato *Sequential*, contendo três camadas densamente conectadas. A primeira camada recebe como entrada um vetor com dimensão igual ao número de atributos disponíveis, seguido por uma camada densa com 128 neurônios e função de ativação ReLU. Esta camada foi regularizada por meio de penalização L1, de forma a reduzir o risco de sobreajuste.

Em seguida, aplicou-se normalização em lote (*Batch Normalization*) e uma camada de *Dropout* com taxa de 60%, visando maior robustez e generalização. A segunda camada oculta é composta por 32 neurônios, também com ativação ReLU e regularização L1, seguida novamente por *Batch Normalization* e *Dropout* de 60%. Por fim, a camada de saída é formada por um único neurônio linear, responsável por produzir a previsão da pontuação final do clube na temporada.

A arquitetura do modelo pode ser resumida da seguinte forma:

- Camada de entrada: vetor de dimensão n (atributos do jogador e do clube).
- Primeira camada oculta: `Dense(128)`, ativação ReLU, regularização L1.
- `Batch Normalization`.
- `Dropout(0.6)`.
- Segunda camada oculta: `Dense(32)`, ativação ReLU, regularização L1.

- `Batch Normalization`.
- `Dropout(0.6)`.
- Camada de saída: `Dense(1)`, ativação linear.

O MLP foi escolhido como base para o modelo de previsão de pontuação no contexto do futebol por reunir propriedades teóricas e práticas que o tornam adequado para lidar com a natureza complexa e não linear dos dados esportivos. Entre as principais razões para essa escolha, destacam-se:

1. **Capacidade de representação universal** — Um MLP com pelo menos uma camada oculta e funções de ativação não lineares é capaz de aproximar qualquer função mensurável (HORNIK, 1991). No contexto do futebol, isso significa que o modelo pode aprender padrões não lineares entre variáveis de desempenho (como gols, desarmes, progressão). Assim, o MLP não depende de hipóteses rígidas sobre a forma das relações entre variáveis, sendo capaz de aprendê-las diretamente dos dados.

2. **Flexibilidade para entradas de alta dimensão** — De acordo com *Goodfellow et. al., 2016*, cada camada de uma rede *feedforward* aprende uma representação do vetor de entrada que torna mais fácil a predição da saída desejada. Essa característica confere ao MLP flexibilidade para lidar com conjuntos de dados de alta dimensionalidade, nos quais múltiplas variáveis interagem de forma não linear. Mesmo com a aplicação prévia da técnica de *Principal Component Analysis* (PCA), que reduz a dimensionalidade de 48 para 20 componentes principais, o problema ainda envolve um espaço vetorial de considerável complexidade. Nesse contexto, o MLP é capaz de explorar as relações não lineares entre os componentes principais, aprendendo representações mais informativas que auxiliam na previsão de desempenho esportivo.

3. **Treinamento eficiente via retropropagação** — O algoritmo de *backpropagation*, apresentado em (GOODFELLOW; BENGIO; COURVILLE, 2016), permite o cálculo eficiente dos gradientes de erro e a atualização simultânea dos pesos da rede por meio de gradiente descendente. Isso viabiliza o treinamento em grandes bases de dados de futebol, reduz o tempo de convergência e possibilita otimizar métricas contínuas.

Além dessas vantagens teóricas, o MLP oferece um equilíbrio entre capacidade de generalização, interpretabilidade e eficiência computacional, características desejáveis em modelos aplicados ao futebol, onde os dados são heterogêneos, ruidosos e interdependentes.

5.4 Otimização

Uma vez identificado o modelo de ML com melhor capacidade de predizer a pontuação final dos times em uma liga a partir das informações estatísticas de seus elencos, inicia-se a segunda etapa do projeto, dedicada à busca de soluções capazes de otimizar esse desempenho. Nessa fase, o problema passa a ser tratado como um processo de otimização sob restrições, no qual diferentes composições de elenco são avaliadas de acordo com sua pontuação estimada pelo modelo de AM.

5.4.1 Cálculo do Valor de Mercado Sintético

Para viabilizar o processo de otimização, torna-se necessário incorporar restrições financeiras que reflitam, ainda que de forma simplificada, as limitações orçamentárias enfrentadas por clubes de futebol em contextos reais. Nesse sentido, foi definida uma métrica sintética de valor de mercado dos jogadores, construída a partir de dois fatores principais: idade e desempenho estatístico agregado.

A Equação 5.1 apresenta formalmente o cálculo adotado para essa métrica.

$$V_i = 1000 \cdot \left(1000 \cdot \max(0, 30 - I_i) + 10 \cdot \sum_{j=1}^{n} x_{ij} \right) \tag{5.1}$$

onde:

- V_i representa o valor de mercado sintético do jogador i;
- I_i corresponde à idade do jogador i;
- x_{ij} representa o valor da j-ésima estatística numérica associada ao jogador i;
- n é o número total de variáveis estatísticas consideradas.

O primeiro termo da equação corresponde a um fator etário, que atribui maior peso a jogadores mais jovens, considerando um limite superior de 30 anos. Jogadores com idade superior a esse limiar não recebem penalização adicional, evitando a atribuição de valores negativos. Essa escolha busca refletir uma característica recorrente do mercado do futebol, no qual atletas mais jovens tendem a apresentar maior potencial de valorização, longevidade esportiva e retorno econômico futuro.

O segundo termo da equação agrega o desempenho estatístico do jogador por meio da soma de suas variáveis numéricas, escalonada por um fator constante. Essa agregação tem como objetivo capturar, de forma simplificada, o impacto global do desempenho esportivo individual, sem privilegiar métricas específicas ou posições em campo.

Por fim, o fator multiplicativo externo é aplicado exclusivamente para reescalonamento da variável, de modo a produzir valores em uma ordem de grandeza mais próxima às estimativas observadas no mercado real. Esse reescalonamento facilita tanto a interpretação dos valores obtidos quanto a utilização da métrica como restrição orçamentária nos modelos de otimização empregados na etapa subsequente.

Dessa forma, jogadores que combinam alto desempenho estatístico com menor idade tendem a apresentar valores de mercado sintético mais elevados, enquanto atletas mais velhos ou com desempenho inferior são associados a custos reduzidos, em consonância com evidências empíricas da literatura que apontam idade e desempenho como fatores centrais na formação do valor econômico de jogadores profissionais. (FRICK, 2007; MÜLLER; SIMONS; WEINMANN, 2017) Ressalta-se que essa métrica não tem como objetivo reproduzir valores reais de mercado, mas sim fornecer uma função custo coerente e monotônica, adequada ao processo de otimização proposto.

A partir da definição dessa função de custo, torna-se possível formular o problema de seleção de jogadores como um problema de otimização combinatória, no qual se busca identificar configurações de elenco que maximizem o desempenho previsto da equipe sob restrições orçamentárias. Nesse contexto, diferentes estratégias heurísticas podem ser empregadas para explorar o espaço de soluções. Como tais métodos frequentemente apresentam componentes estocásticos e podem produzir resultados distintos entre execuções, sua avaliação requer múltiplas repetições experimentais e análise estatística apropriada. Nesse sentido, conforme discutido por (DERRAC et al., 2011), testes estatísticos não paramétricos são amplamente recomendados na comparação de algoritmos heurísticos, pois não pressupõem normalidade na distribuição dos resultados obtidos. A seguir, descreve-se os métodos de otimização utilizados neste trabalho.

5.4.2 Algoritmos Gulosos para Seleção Iterativa de Jogadores

Com o objetivo de explorar possíveis melhorias na composição do elenco, foram implementados algoritmos gulosos responsáveis por selecionar iterativamente jogadores a serem contratados.

O método parte do elenco atual da equipe e constrói gradualmente um conjunto de reforços. Em cada iteração, o algoritmo percorre a base de dados de

jogadores e considera aqueles que atuaram na temporada imediatamente anterior à temporada analisada. Durante essa varredura, são desconsiderados jogadores que já pertencem ao elenco atual da equipe ou que já foram previamente selecionados em iterações anteriores do próprio algoritmo.

Além disso, cada possível contratação é submetida à restrição orçamentária. O valor de mercado do atleta, registrado na temporada anterior, é utilizado para verificar se a contratação pode ser realizada dentro do orçamento disponível. Apenas jogadores que respeitam essa restrição são avaliados.

A qualidade de cada possível contratação é avaliada por meio da função objetivo definida neste trabalho, implementada como o modelo de AM responsável por estimar a pontuação esperada da equipe na temporada considerada. Para manter constante o tamanho do elenco avaliado, a inclusão de novos jogadores é simulada removendo-se, da formação original, a mesma quantidade de atletas com menor minutagem na temporada analisada.

O processo é repetido até que seja atingido um número máximo de contratações, definido por um parâmetro k, ou até que não existam mais jogadores viáveis dentro da restrição orçamentária. Sempre que uma contratação é aceita, o valor de mercado do jogador selecionado é subtraído do orçamento disponível, atualizando a restrição orçamentária para as iterações seguintes. Como característica fundamental da estratégia gulosa, as decisões tomadas em cada iteração são definitivas: uma vez que um jogador é selecionado e adicionado ao conjunto de reforços, essa escolha não é revista nas iterações subsequentes.

Foram implementadas duas variações desse algoritmo. Ambas possuem exatamente a mesma estrutura e seguem o mesmo procedimento descrito anteriormente, diferindo apenas na forma como a melhoria da função objetivo é aceita em cada iteração. Na versão *first-improvement*, os jogadores são avaliados sequencialmente durante a varredura da base e a primeira contratação que resulta em melhoria da função objetivo é imediatamente aceita. Já na versão *best-improvement*,

todos os jogadores viáveis são avaliados em cada iteração e seleciona-se aquele cuja inclusão produz o maior aumento na função objetivo.

Os pseudocódigos dos algoritmos gulosos utilizados são apresentados a seguir.

Algorithm 2 Algoritmo Guloso com estratégia First Improvement

```
Obter lista de jogadores candidatos da temporada anterior
Obter elenco atual do time alvo
Ordenar jogadores do elenco atual por menor minutagem
S ← ∅                                                    ▷ novas contratações
calcular pontuação atual
for i = 1 até size do
    for cada jogador candidato p do
        if p não pertence ao elenco atual e cabe no orçamento then
            Simular substituição do jogador de menor minutagem por p
            Calcular nova pontuação prevista
            if pontuação melhora then
                adicionar p em S
                atualizar orçamento
                atualizar pontuação
                break                                    ▷ first improvement
            end if
        end if
    end for
    if nenhuma melhora encontrada then
        break
    end if
end for
return conjunto de contratações e pontuação prevista
```

5.4.3 Algoritmos de Busca Local para Otimização do Elenco

Além das heurísticas gulosas descritas na subseção anterior, foram implementados algoritmos baseados em busca local com o objetivo de explorar de forma mais ampla o espaço de soluções possíveis para o problema de montagem de elenco.

Algorithm 3 Algoritmo Guloso com estratégia Best Improvement

```
Obter jogadores candidatos da temporada anterior
Obter elenco atual do time alvo
Ordenar jogadores do elenco atual por menor minutagem
S ← ∅                                                    ▷ conjunto de contratações
calcular pontuação inicial
for i = 1 até size do
    best_player ← ∅
    best_score ← pontuação atual
    for cada jogador candidato p do
        if p não pertence ao elenco atual e cabe no orçamento then
            Simular substituição do jogador de menor minutagem por p
            Calcular pontuação prevista
            if pontuação > best_score then
                atualizar best_player e best_score
            end if
        end if
    end for
    if best_player ≠ ∅ then
        adicionar best_player ao conjunto de contratações
        atualizar orçamento
        atualizar pontuação
    else
        break
    end if
end for
return contratações e pontuação prevista
```

Enquanto os algoritmos gulosos constroem o conjunto de contratações de forma incremental, selecionando jogadores individualmente a cada iteração, a estratégia de busca local parte de uma solução inicial completa e passa a realizar modificações estruturais nessa solução ao longo do processo de otimização.

Cada solução é representada por um conjunto de jogadores candidatos à contratação cuja cardinalidade é limitada por um parâmetro k, que define o número máximo de contratações consideradas. A avaliação de cada solução é realizada por meio da mesma função objetivo utilizada nos algoritmos gulosos, isto é, o modelo de AM desenvolvido neste para estimar a pontuação esperada da equipe ao final da temporada.

Assim como na abordagem gulosa, a inclusão de jogadores no elenco é simulada mantendo constante o tamanho da formação avaliada. Para isso, sempre que um conjunto de contratações é considerado, remove-se da formação original a mesma quantidade de atletas com menor minutagem na temporada analisada.

A solução inicial é construída selecionando aleatoriamente jogadores da temporada imediatamente anterior à temporada analisada e respeitando a restrição orçamentária estabelecida. Durante essa etapa, são desconsiderados jogadores que já pertencem ao elenco atual da equipe. O processo de seleção inicial continua até que seja atingido o limite máximo de contratações definido pelo parâmetro k ou até que o orçamento disponível não permita novas adições.

Uma vez obtida a solução inicial viável, o algoritmo passa a explorar sua vizinhança por meio de operações de substituição unitária (*swap*). Em cada vizinho gerado, um jogador do conjunto atual de contratações é substituído por outro jogador elegível ainda não selecionado. A substituição somente é considerada factível quando o valor de mercado do novo jogador, descontado o valor do atleta removido, respeita a restrição orçamentária estabelecida.

Uma vantagem importante da abordagem de busca local em relação às heurísticas gulosas é que a avaliação da função objetivo passa a considerar direta-

mente o conjunto completo de contratações selecionadas. Enquanto os algoritmos gulosos analisam o impacto marginal da adição de um único jogador por vez, a busca local avalia modificações em uma solução já formada, permitindo que possíveis interações entre os jogadores selecionados sejam levadas em conta pelo modelo de avaliação. Dessa forma, correlações positivas ou negativas entre atletas — capturadas implicitamente pelo modelo de AM utilizado como função objetivo — podem influenciar a escolha das substituições realizadas durante o processo de otimização.

De maneira análoga ao caso dos algoritmos gulosos, duas variações da busca local foram implementadas, diferenciando-se exclusivamente pelo critério de aceitação das melhorias encontradas. Na versão *first-improvement*, as substituições são avaliadas sequencialmente e a primeira troca que produz melhoria na função objetivo é imediatamente aceita, reiniciando-se a exploração da vizinhança a partir da nova solução. Já na versão *best-improvement*, todas as substituições possíveis na vizinhança são avaliadas antes da atualização da solução corrente, sendo então aplicada a troca que produz o maior aumento na função objetivo.

O processo iterativo é encerrado quando nenhuma substituição viável resulta em melhoria adicional na função objetivo, caracterizando a convergência para um ótimo local no espaço de soluções explorado.

Os pseudocódigos dos algoritmos de busca local utilizados são apresentados a seguir.

Algorithm 4 Busca local com estratégia First Improvement

```
Obter elenco atual do time alvo
Ordenar elenco atual por menor minutagem
Selecionar candidatos da temporada anterior
Gerar uma solução inicial viável respeitando o orçamento
Calcular a pontuação inicial da solução
while houver melhoria do
    for cada posição de contratação i do
        for cada jogador candidato p do
            if p não pertence ao elenco atual, não está na solução e cabe no
orçamento then
                Substituir temporariamente o jogador da posição i por p
                Calcular a nova pontuação prevista
                if a pontuação melhorar then
                    Aceitar a troca
                    Atualizar orçamento e solução atual
                    break                                   ▷ first improvement
                end if
            end if
        end for
        if uma melhoria foi aceita then
            break
        end if
    end for
end while
return solução final, pontuação prevista e orçamento restante
```

Algorithm 5 Busca local com estratégia Best Improvement

```
Obter elenco atual do time alvo
Ordenar elenco atual por menor minutagem
Selecionar candidatos da temporada anterior
Gerar uma solução inicial viável respeitando o orçamento
Calcular a pontuação inicial da solução
while houver melhoria do
    Inicializar a melhor solução vizinha com a solução atual
    for cada posição de contratação i do
        for cada jogador candidato p do
            if p não pertence ao elenco atual, não está na solução e cabe no
orçamento then
                Substituir temporariamente o jogador da posição i por p
                Calcular a nova pontuação prevista
                if a pontuação melhorar a melhor vizinha encontrada then
                    Atualizar a melhor solução vizinha
                end if
            end if
        end for
    end for
    if foi encontrada uma vizinhança melhor then
        Aceitar a melhor troca encontrada
        Atualizar orçamento e solução atual
    else
        break
    end if
end while
return solução final, pontuação prevista e orçamento restante
```

6 RESULTADOS E DISCUSSÕES

6.1 Modelos de Aprendizado de Máquinas

O melhor modelo de AM desenvolvido neste estudo apresentou acurácia máxima de aproximadamente 60% na tarefa de prever a pontuação final de um time em sua liga nacional, utilizando como base os dados estatísticos de desempenho individual de seus jogadores na temporada anterior.

Para avaliar o desempenho dos diferentes algoritmos aplicados à tarefa de previsão, foram calculadas métricas amplamente utilizadas em problemas de regressão, incluindo o coeficiente de determinação (R^2), o erro absoluto médio (MAE), o erro quadrático médio (MSE), a raiz do erro quadrático médio (RMSE), o erro percentual absoluto médio (MAPE) e o tempo de processamento de cada modelo. A Tabela 6.1 apresenta um resumo comparativo dos resultados obtidos, permitindo analisar tanto a qualidade preditiva quanto o custo computacional de cada abordagem.

Tabela 6.1 – Comparação de desempenho entre os modelos de Machine Learning avaliados no conjunto de testes

Modelo	**R^2**	**MAE**	**MSE**	**RMSE**	**MAPE**	**Tempo (s)**
RL	0.41	11.48	191.78	13.85	0.2580	2.15
RF	0.28	12.05	234.05	15.30	0.2690	423.13
XGBoost	0.41	11.48	191.78	13.85	0.2580	2.86
MLP	0.55	11.08	170.42	13.05	0.2424	161.68

Fonte: Elaborado pelo autor.

Os resultados apresentados na Tabela 6.1 evidenciam um comportamento consistente entre os algoritmos avaliados, com desempenho moderado na tarefa de previsão da pontuação final dos times. Embora a acurácia máxima tenha alcançado aproximadamente 55%, observa-se que todos os modelos enfrentam limitações inerentes à natureza imprevisível, ruidosa e não linear do problema. A pontuação de um time ao longo de uma temporada é influenciada também por fato-

res não refletidos nas estatísticas individuais de jogadores, como variações táticas, mudanças de treinador, lesões, contexto competitivo e dinâmica coletiva, o que naturalmente reduz o teto preditivo esperável.

Entre as abordagens testadas, a Rede Neural Multicamadas (MLP) apresentou o melhor equilíbrio entre qualidade preditiva e estabilidade, alcançando o maior valor de R^2 (0,55) e os menores erros MAE, MSE, RMSE e MAPE. Esse resultado sugere que a arquitetura densa foi mais capaz de capturar as relações não lineares presentes nos componentes principais gerados pelo PCA, aproveitando estruturas de dependência que modelos lineares ou baseados em árvores não exploram de maneira tão eficaz.

O desempenho mais modesto do modelo de *Random Forest* (RF) indica que, apesar de ser um método robusto e amplamente utilizado em tarefas de AM, quando reduzido por PCA, funcionou de forma mais limitada. Além disso, o custo computacional do RF foi significativamente superior, tornando-o menos atrativo para cenários de larga escala ou simulações repetidas de montagem de elenco.

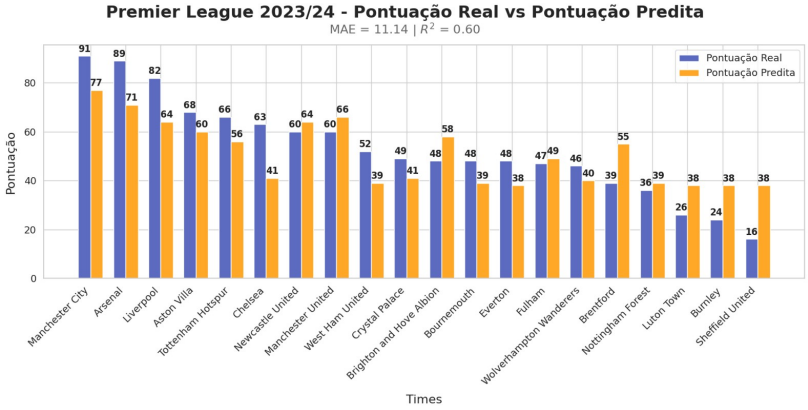
O modelo de Regressão Linear (RL) apresentou desempenho comparável ao *XGBoost*, demonstrando que uma parte da variabilidade explicável está capturada em relações aproximadamente lineares entre os componentes principais do PCA e a pontuação final dos times. No entanto, o ganho obtido pela MLP reforça que há padrões não lineares relevantes que justificam a adoção de modelos mais complexos.

6.1.1 Predição do Modelo MLP

Com o intuito de compreender a predição realizada pelo melhor modelo (MLP) foi realizada uma análise visual da relação entre as pontuações reais dos clubes da Premier League na temporada e as pontuações previstas pelo modelo.

A Figura **??** apresenta o gráfico de comparação entre as pontuações reais e previstas pelo modelo MLP.

Figura 6.1 – Relação entre pontuações reais e previstas pelo modelo MLP na Premier League.



Os resultados apresentados na Figura **??** indicam que o modelo MLP apresenta maior precisão ao estimar as pontuações de clubes que terminaram a temporada com valores próximos da média da liga. Times posicionados no centro da distribuição como Fulham, Manchester United e Newcastle United tendem a ter suas pontuações previstas com menor erro, comportamento comum em modelos preditivos sob alta incerteza: quando os atributos disponíveis não capturam toda a variabilidade do fenômeno, as previsões convergem naturalmente para valores intermediários. Esse resultado alinha-se com a literatura clássica, segundo a qual modelos preditivos em ambientes de alta variabilidade, como é o caso do futebol, tendem a gerar funções de saída menos extremas e com menor amplitude, aproximando-se de valores centrais, devido a mecanismos de *shrinkage* e ao balanço entre viéses e variância discutidos por Bishop (BISHOP, 2006). Esse comportamento é análogo ao fenômeno estatístico conhecido como regressão à média.

Dessa forma, observa-se que o modelo MLP é capaz de capturar tendências gerais de desempenho dos clubes com base nas estatísticas individuais de seus jogadores, porém nota-se uma subestimação sistemática das equipes de desempenho elevado, como Manchester City, Arsenal e Liverpool, e uma superestimação das equipes de baixo desempenho, como Sheffield United, Burnley e Luton Town.

6.2 Resultados da Otimização de Montagem de Elenco

Uma vez que os algoritmos de otimização utilizam o modelo de AM como função objetivo, a precisão final das soluções depende diretamente das previsões produzidas por esse modelo. Dessa forma, tanto o algoritmo guloso quanto o método de busca local operam sobre a mesma função objetivo, diferenciando-se apenas pela estratégia utilizada para explorar o espaço de soluções. Assim, a comparação entre os métodos concentra-se principalmente na qualidade das soluções encontradas e no custo computacional necessário para alcançá-las.

Para avaliar o desempenho dos algoritmos, ambos foram executados sob as mesmas condições experimentais, utilizando os mesmos limites de tempo, número máximo de avaliações da função objetivo e parâmetros de busca. Cada método foi executado 15 vezes com diferentes sementes aleatórias, permitindo observar o comportamento médio das soluções obtidas.

A Tabela 6.2 apresenta os resultados agregados obtidos pelos dois algoritmos. Foram considerados como métricas de comparação a pontuação prevista média obtida pelas soluções encontradas, o desvio padrão das pontuações, a melhor solução encontrada, o tempo médio de execução e o número médio de avaliações da função objetivo.

Tabela 6.2 – Comparação entre os algoritmos de otimização

Algoritmo	Score médio	Desvio padrão	Melhor score	Tempo médio (s)	Avaliações médias	Desv
Busca Local	72.675	0.046	72.747	120.173	429.80	
Guloso	72.704	0.028	72.736	120.211	422.67	

Fonte: Elaboração própria.

Em relação ao custo computacional, os dois métodos apresentaram tempos médios de execução praticamente idênticos, uma vez que ambos operam sob o mesmo limite de tempo imposto durante o experimento. O número médio de avaliações da função objetivo também se mostrou semelhante entre os métodos, embora o algoritmo guloso tenha realizado ligeiramente menos avaliações em média.

Em conjunto, esses resultados indicam que, para o problema analisado, o algoritmo guloso é capaz de alcançar soluções de qualidade equivalente àquelas obtidas pela busca local, utilizando um número similar de avaliações da função objetivo. Isso sugere que, apesar de sua simplicidade, a abordagem gulosa pode representar uma alternativa eficiente para a otimização da composição de elencos no contexto proposto.

6.3 Conclusões

Este trabalho apresentou uma abordagem integrada de AM e PO para apoiar o processo de montagem de elencos no futebol profissional. Inicialmente, foi desenvolvido um modelo preditivo capaz de estimar o desempenho esperado de equipes a partir das características individuais dos jogadores. Os resultados obtidos indicam que o modelo proposto alcança desempenho aproximadamente 55% superior ao baseline adotado, evidenciando sua capacidade de capturar padrões relevantes nos dados e produzir estimativas mais precisas de desempenho coletivo.

A partir dessas previsões, o modelo foi utilizado como função objetivo em um problema de otimização voltado à seleção de jogadores sob restrições orçamentárias. Para explorar o espaço de soluções, foram implementadas duas heurísticas: um algoritmo guloso baseado em substituições iterativas e um método de busca local. *Inserir resultados dos algoritmos de PO*

Do ponto de vista prático, os resultados obtidos indicam que a abordagem proposta pode contribuir para tornar o processo de tomada de decisão na montagem de elencos mais estruturado e orientado por dados. No contexto do futebol brasileiro, onde decisões de contratação frequentemente são influenciadas por fatores subjetivos ou avaliações intuitivas, ferramentas baseadas em análise quantitativa podem oferecer suporte adicional à identificação de jogadores que se encaixem de forma mais eficiente em determinadas estruturas de equipe, respeitando restrições financeiras.

Assim, este trabalho demonstra que a integração entre técnicas de AM e métodos de otimização representa um caminho promissor para apoiar processos de recrutamento e planejamento esportivo. Ao transformar dados de desempenho em estimativas de impacto coletivo e utilizá-las em modelos de decisão, torna-se possível reduzir a dependência exclusiva da intuição na escolha de atletas e avançar em direção a abordagens mais analíticas na gestão do futebol.

REFERÊNCIAS

ABIDIN, D. A case study on player selection and team formation in football with machine learning. **Turkish Journal of Electrical Engineering and Computer Sciences**, v. 29, n. 3, 2021. Disponível em: <https://journals.tubitak.gov.tr/elektrik/vol29/iss3/23>.

ALMULLA, J. et al. Soccernet: A gated recurrent unit-based model to predict soccer match winners. **PLoS ONE**, ago. 2023. Disponível em: <https://doi.org/10.1371/journal.pone.0288933>.

AWADALLAH, A. A.; KHANDELWAL, R. Football match prediction using deep learning (recurrent neural network). 2020. CS230 Deep Learning Project. Disponível em: <https://github.com/raghav68/CS230DLProject>.

BABOOTA, R.; KAUR, H. Predictive analysis and modelling football results using machine learning approach for english premier league. **International Journal of Forecasting**, v. 35, n. 2, p. 741–755, 2019. Disponível em: <https://doi.org/10.1016/j.ijforecast.2018.01.003>.

BBC Sport. **Moises Caicedo transfer news: Chelsea agree £115m deal for Brighton midfielder**. 2023. Accessed: 28 Jan. 2026. Disponível em: <https://www.bbc.com/sport/football/66491106>.

Beautiful Soup Documentation. **Beautiful Soup Documentation**. 2024. Acessado em: 10 fev. 2025. Disponível em: <https://beautiful-soup-4.readthedocs.io/en/latest/>.

BECKER, A.; SUN, X. A. An analytical approach for fantasy football draft and lineup management. **Journal of Quantitative Analysis in Sports**, De Gruyter, v. 11, n. 1, p. 13–27, 2015.

BERTSIMAS, D.; KALLUS, N. From predictive to prescriptive analytics. **Management Science**, INFORMS, v. 65, n. 3, p. 1025–1044, 2019.

BISHOP, C. M. **Pattern Recognition and Machine Learning**. New York: Springer, 2006. (Information Science and Statistics). ISBN 978-0-387-31073-2.

BOON, B. H.; SIERKSMA, G. Team formation: Matching quality supply and quality demand. **European Journal of Operational Research**, Elsevier, v. 148, n. 1, p. 71–82, 2003.

BREFELD, U. et al. (Ed.). **Machine Learning and Data Mining for Sports Analytics**: 10th international workshop, mlsa 2023, turin, italy, september 18, 2023, revised selected papers. [S.l.]: Springer Nature Switzerland, 2024. v. 2035. (Communications in Computer and Information Science, v. 2035). ISBN 978-3-031-53832-2.

CARMICHAEL, F.; MCHALE, I.; THOMAS, D. Maintaining market position: Team performance, revenue and wage expenditure in the english premier league. **Bulletin of Economic Research**, v. 63, n. 4, 2011.

CHMAIT, N.; WESTERBEEK, H. Artificial intelligence and machine learning in sport research: An introduction for non-data scientists. **Frontiers in Sports and Active Living**, v. 3, p. 682287, 2021. Disponível em: <https://doi.org/10.3389/fspor.2021.682287>.

CLARKE, S. R.; NORMAN, J. M. When to rush a behind in australian rules football: A dynamic programming approach. **Journal of the Operational Research Society**, Palgrave Macmillan, v. 49, n. 5, p. 530–536, 1998.

CNN Brasil. **Moisés Caicedo no Chelsea: de desconhecido a jogador mais caro do futebol inglês**. 2023. Accessed: 28 Jan. 2026. Disponível em: <https://www.cnnbrasil.com.br/esportes/futebol/futebol-internacional/moises-caicedo-no-chelsea-de-desconhecido-a-jogador-mais-caro-do-futebol-ingles/>.

CORMEN, T. H. et al. **Introduction to Algorithms**. 3rd. ed. Cambridge, MA: The MIT Press, 2009. ISBN 978-0262033848.

DERRAC, J. et al. A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms. **Swarm and Evolutionary Computation**, Elsevier, v. 1, n. 1, p. 3–18, 2011. ISSN 2210-6502.

FBref. **FBref: Football Statistics and History**. 2026. Acesso em: 2024. Disponível em: <https://fbref.com/>.

FRICK, B. The football players' labor market: Empirical evidence from the major european leagues. **Scottish Journal of Political Economy**, Blackwell Publishing, v. 54, n. 3, p. 422–446, 2007.

GENDREAU, M.; POTVIN, J.-Y. (Ed.). **Handbook of Metaheuristics**. 2. ed. New York, NY: Springer, 2010. (International Series in Operations Research & Management Science).

GERCHAK, Y. Operations research in sports. In: **Handbooks in Operations Research & Management Science**. [S.l.]: Elsevier Science B.V., 1994. v. 6, p. 507–512.

GLOBO Esporte. **Mercado de transferências bate recorde em 2023: R$ 36,6 bilhões**. 2023. Acesso em: 11 fev. 2025. Disponível em: <https://ge.globo.com/futebol/futebol-internacional/noticia/2023/09/08/mercado-de-transferencias-bate-recorde-em-2023-r-366-bilhoes.ghtml>.

GLOBO Esporte. **Janela da Série A bate marca de R$ 1 bilhão em reforços; veja quem mais gastou**. 2025. Acesso em: 24 fev. 2025. Disponível em: <https://ge.globo.com/espiao-estatistico/noticia/2025/02/06/janela-da-serie-a-bate-marca-de-r-1-bilhao-em-reforcos-veja-quem-mais-gastou.ghtml>.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. **Deep Learning**. MIT Press, 2016. ISBN 978-0-262-03561-3. Disponível em: <https://www.deeplearningbook.org>.

GRAHAM, I. **How to Win the Premier League**. [S.l.]: Century, 2024.

GUYON, I. et al. (Ed.). **Feature Extraction: Foundations and Applications**. Berlin Heidelberg: Springer, 2006. v. 207. (Studies in Fuzziness and Soft Computing, v. 207). ISBN 978-3-540-35487-1.

GéRON, A. **Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow**. 2nd. ed. [S.l.]: O'Reilly Media, 2019.

HEROLD, M.; MEMMERT, D.; PERL, J. Machine learning in men's professional football: Current applications and future directions for improving attacking play. **International Journal of Sports Science & Coaching**, SAGE Publications, v. 14, n. 6, p. 798–817, 2019.

HILLIER, F. S.; LIEBERMAN, G. J. **Introduction to Operations Research**. 7. ed. New York: McGraw-Hill, 2001. ISBN 978-0072416186.

HIROTSU, N.; WRIGHT, M. Using a markov process model of an association football match to determine the optimal timing of substitution and tactical decisions. **Journal of the Operational Research Society**, v. 53, p. 88–96, 2002.

HORNIK, K. Approximation capabilities of multilayer feedforward networks. **Neural Networks**, v. 4, p. 251–257, 1991.

HORVAT, T.; JOB, J. The use of machine learning in sport outcome prediction: A review. **WIREs Data Mining and Knowledge Discovery**, Wiley, v. 10, n. 6, p. e1382, 2020.

JORDAN, M. I.; MITCHELL, T. M. Machine learning: Trends, perspectives, and prospects. **Science**, v. 349, n. 6245, p. 255–260, 2015.

JOSEPH, A.; FENTON, N. E.; NEIL, M. Predicting football results using bayesian nets and other machine learning techniques. **Knowledge-Based Systems**, v. 19, 2006.

KELLEHER, J. D.; NAMEE, B. M.; D'ARCY, A. **Fundamentals of Machine Learning for Predictive Data Analytics: Algorithms, Worked Examples, and Case Studies**. Cambridge, MA: MIT Press, 2015. ISBN 978-0262029445.

KENDALL, G.; PARKES, A.; SPOERER, K. A survey of np-complete scheduling problems. **Journal of Scheduling**, Springer, v. 13, n. 3, p. 257–276, 2010.

KITE, C. S.; NEVILL, A. The predictors and determinants of inter-seasonal success in a professional soccer team. **Journal of Human Kinetics**, v. 58, p. 157–167, 2017.

LAGO-BALLESTEROS, J.; LAGO-PEñAS, C. Performance in team sports: Identifying the keys to success in soccer. **Journal of Human Kinetics**, v. 25, p. 85–91, 2010.

LIU, H. et al. Feature selection: An ever evolving frontier in data mining. In: LAWRENCE, N. (Ed.). **JMLR: Workshop and Conference Proceedings - The Fourth Workshop on Feature Selection in Data Mining (FSDM 2010)**. [S.l.]: JMLR Workshop and Conference Proceedings, 2010. v. 10, p. 4–13.

MAHESH, B. Machine learning algorithms - a review. **International Journal of Science and Research (IJSR)**, v. 9, p. 381–386, jan. 2020. Disponível em: <https://doi.org/10.21275/ART20203995>.

MEDRI, W.; YOTSUMOTO, A. **Pesquisa Operacional na Tomada de Decisão**. [S.l.: s.n.], 2009.

MÜLLER, O.; SIMONS, A.; WEINMANN, M. Beyond crowd judgments: Data-driven estimation of market value in association football. **European Journal of Operational Research**, Elsevier, v. 263, n. 1, p. 273–287, 2017.

NOBLE, W. S. What is a support vector machine? **Nature Biotechnology**, Nature Publishing Group, v. 24, n. 12, p. 1565–1567, 2006.

OLLEY, J. **How Brighton's transfer mastery broke Premier League profit record**. ESPN, 2023. Acesso em: 11 fev. 2025. Disponível em: <https://www.espn.com/soccer/story/_/id/40069726/how-brightons-transfer-mastery-broke-premier-league-profit-record>.

PANTUSO, G. The football team composition problem: A stochastic programming approach. **Journal of Quantitative Analysis in Sports**, 2017.

QADER, M. A. et al. A methodology for football players selection problem based on multi-measurements criteria analysis. **Measurement**, v. 111, p. 38–50, 2017.

RICO-GONZÁLEZ, M. et al. Machine learning application in soccer: a systematic review. **Biology of Sport**, Institute of Sport, v. 40, n. 1, p. 249–263, 2023.

RODRIGUES, F.; PINTO, A. Prediction of football match results with machine learning. **Procedia Computer Science**, v. 204, p. 463–470, jan. 2023. Disponível em: <https://doi.org/10.1016/j.procs.2022.08.057>.

TAVANA, M. et al. A fuzzy inference system with application to player selection and team formation in multi-player sports. **Sport Management Review**, v. 16, p. 97–110, 2013.

TOTTY, E. S.; OWENS, M. F. Salary caps and competitive balance in professional sports leagues. **Journal for Economic Educators**, v. 11, n. 2, p. 46–56, 2011.

WITTEN, I. H. et al. **Data Mining: Practical Machine Learning Tools and Techniques**. 4. ed. Burlington, MA: Morgan Kaufmann, 2016. ISBN 978-0128042915.

WRIGHT, M. B. 50 years of or in sport. **Journal of the Operational Research Society**, Palgrave Macmillan, v. 60, n. S1, p. S161–S168, 2009.

A PSEUDOALGORITMOS COMPLETOS DOS ALGORITMOS GULOSOS

Neste apêndice são apresentados os pseudoalgoritmos completos utilizados na implementação dos métodos gulosos descritos no Capítulo 5.

Algoritmo 1 – Algoritmo Guloso First-Improvement para Seleção de Jogadores

1: **function** GULOSOFIRST($team, season, player_attributes, player_attributes_PCA, points, size, budget$)
Função auxiliar PREVIOUSSEASON(s):
2: separar s em y_1 e y_2
3: **return** $(y_1 - 1) - (y_2 - 1)$
4: $prev_season \leftarrow$ PREVIOUSSEASON($season$)
5: $player_names \leftarrow$ jogadores únicos em $player_attributes$ com $Season = prev_season$
6: $current_team_df \leftarrow$ registros de $player_attributes$ tais que $Team = team$ e $Season = season$
7: $current_team_players \leftarrow$ conjunto de jogadores únicos em $current_team_df$
8: definir coluna de minutagem como $Min.$ ou Min
9: $current_team_roster \leftarrow$ jogadores únicos em $current_team_df$
10: $lowest_minutes_players \leftarrow$ jogadores de $current_team_df$ ordenados por minutagem crescente
Função auxiliar PREVE($new_players$):
11: $n_remove \leftarrow |new_players|$
12: $removed_players \leftarrow$ primeiros n_remove jogadores de $lowest_minutes_players$
13: $simulated_roster \leftarrow$ jogadores de $current_team_roster$ que não pertencem a $removed_players$, unidos a $new_players$
14: **return** PREVE_PONTOS($team, season, simulated_roster, player_attributes, player_attributes_PCA, points$)
15: $current_new_players \leftarrow \emptyset$
16: $current_score \leftarrow$ PREVE($current_new_players$)

```
    improved ← verdadeiro
    for i = 1 até size do
        if improved = falso then
            interromper o laço
        end if
        improved ← falso
        for cada jogador p em player_names do
            if p ∈ current_new_players then
                passar ao próximo jogador
            end if
            if p ∈ current_team_players then
                passar ao próximo jogador
            end if
            row ← registro de player_attributes tal que Player = p e Season =
prev_season
            if row vazio then
                passar ao próximo jogador
            end if
            p_price ← row.Valor_Mercado
            if p_price > budget then
                passar ao próximo jogador
            end if
            prev ← PREVE(current_new_players ∪ {p})
            if prev > current_score then
                adicionar p em current_new_players
                current_score ← prev
                budget ← budget − p_price
                improved ← verdadeiro
```

```
44:                 interrompe o laço
45:             end if
46:         end for
47:     end for
48:     return current_new_players, current_score, budget
49: end function
```

Algoritmo 2 – Algoritmo Guloso Best-Improvement para Seleção de Jogadores

```
1: function GULOSOBEST(team, season, player_attributes, player_attributes_PCA, points, size, budget)
   Função auxiliar PREVIOUSSEASON(s):
2:     separar s em y_1 e y_2
3:     return (y_1 − 1) − (y_2 − 1)
4:     prev_season ← PREVIOUSSEASON(season)
5:     player_names ← jogadores únicos em player_attributes com Season =
   prev_season
6:     current_team_df ← registros de player_attributes tais que Team = team
   e Season = season
7:     current_team_players ← conjunto de jogadores únicos em current_team_df
8:     definir coluna de minutagem como Min. ou Min
9:     current_team_roster ← jogadores únicos em current_team_df
10:    lowest_minutes_players ← jogadores de current_team_df ordenados por
   minutagem crescente
   Função auxiliar PREVE(new_players):
11:    n_remove ← |new_players|
12:    removed_players ← primeiros n_remove jogadores de lowest_minutes_players
13:    simulated_roster ← jogadores de current_team_roster que não perten-
   cem a removed_players, unidos a new_players
14:    return PREVE_PONTOS(team, season, simulated_roster, player_attributes,
        player_attributes_PCA, points)
```

```
    current_new_players ← ∅
    current_score ← PREVE(current_new_players)
    improved ← verdadeiro
    for i = 1 até size do
        if improved = falso then
            interromper o laço
        end if
        improved ← falso
        best_p ← nulo
        best_price ← nulo
        best_score ← current_score
        for cada jogador p em player_names do
            if p ∈ current_new_players then
                passar ao próximo jogador
            end if
            if p ∈ current_team_players then
                passar ao próximo jogador
            end if
            row ← registro de player_attributes tal que Player = p e Season =
prev_season
            if row vazio then
                passar ao próximo jogador
            end if
            p_price ← row.Valor_Mercado
            if p_price > budget then
                passar ao próximo jogador
            end if
            prev ← PREVE(current_new_players ∪ {p})
```

```
42:             if prev > best_score then
43:                 best_score ← prev
44:                 best_p ← p
45:                 best_price ← p_price
46:             end if
47:         end for
48:         if best_p ≠ nulo then
49:             adicionar best_p em current_new_players
50:             current_score ← best_score
51:             budget ← budget − best_price
52:             improved ← verdadeiro
53:         end if
54:     end for
55:     return current_new_players, current_score, budget
56: end function
```

B PSEUDOALGORITMOS COMPLETOS DOS ALGORITMOS DE BUSCA LOCAL

Neste apêndice são apresentados os pseudoalgoritmos completos utilizados na implementação dos métodos de busca local descritos no Capítulo 5.

Algoritmo 3 – Busca Local First-Improvement para Seleção de Jogadores

1: **function** BUSCALOCALFIRST($team, season, player_attributes, player_attributes_PCA, points, size, budge$
Função auxiliar PREVIOUSSEASON(s):
2: separar s em y_1 e y_2
3: **return** $(y_1 - 1) - (y_2 - 1)$
4: $prev_season \leftarrow$ PREVIOUSSEASON($season$)
5: $current_team_df \leftarrow$ registros de $player_attributes$ tais que $Team = team$ e $Season = season$
6: $current_team_players \leftarrow$ conjunto de jogadores únicos em $current_team_df$
7: definir coluna de minutagem como $Min.$ ou Min
8: $current_team_roster \leftarrow$ jogadores únicos em $current_team_df$
9: $lowest_minutes_players \leftarrow$ jogadores de $current_team_df$ ordenados por minutagem crescente
Função auxiliar PREVE($new_players$):
10: $n_remove \leftarrow |new_players|$
11: $removed_players \leftarrow$ primeiros n_remove jogadores de $lowest_minutes_players$
12: $simulated_roster \leftarrow$ jogadores de $current_team_roster$ que não pertencem a $removed_players$, unidos a $new_players$
13: **return** PREVE_PONTOS($team, season, simulated_roster, player_attributes, player_attributes_PCA, points$)
14: $base_players \leftarrow$ jogadores únicos em $player_attributes$ com $Season = prev_season$
15: $player_names \leftarrow$ amostra aleatória de até 100 jogadores de $base_players$

```
    price_map ← dicionário que associa cada jogador de prev_season ao seu Valor_Mercado
    feasible_pool ← jogadores em player_names tais que não pertencem a current_team_players e existem em price_map
    embaralhar feasible_pool
    current_new_players ← ∅
    remaining_budget ← budget
    for cada jogador p em feasible_pool do
        if |current_new_players| ≥ size then
            interromper o laço
        end if
        p_price ← price_map[p]
        if p_price ≤ remaining_budget then
            adicionar p em current_new_players
            remaining_budget ← remaining_budget − p_price
        end if
    end for
    current_score ← PREVE(current_new_players)
    improvement ← verdadeiro
    while improvement = verdadeiro do
        improvement ← falso
        for i = 1 até |current_new_players| do
            new_players ← cópia de current_new_players
            for cada jogador p em player_names do
                if p ∈ current_new_players then
                    passar ao próximo jogador
                end if
                if p ∈ current_team_players then
```

```
                passar ao próximo jogador
            end if
            if p ∉ price_map then
                passar ao próximo jogador
            end if
            p_price ← price_map[p]
            old_player ← current_new_players[i]
            old_price ← price_map[old_player] ou 0 se ausente
            if p_price > remaining_budget + old_price then
                passar ao próximo jogador
            end if
            substituir o jogador da posição i em new_players por p
            new_score ← PREVE(new_players)
            if new_score > current_score then
                current_score ← new_score
                current_new_players ← new_players
                remaining_budget ← remaining_budget + old_price − p_price
                improvement ← verdadeiro
                interromper o laço
            end if
        end for
        if improvement = verdadeiro then
            interromper o laço
        end if
    end for
end while
return current_new_players, current_score, remaining_budget
end function
```

Algoritmo 4 – Busca Local Best-Improvement para Seleção de Jogadores

1: **function** BUSCALOCALBEST($team, season, player_attributes, player_attributes_PCA, points, size, budge$

Função auxiliar PREVIOUSSEASON(s):

2: separar s em y_1 e y_2

3: **return** $(y_1 - 1) - (y_2 - 1)$

4: $prev_season \leftarrow$ PREVIOUSSEASON($season$)

5: $current_team_df \leftarrow$ registros de $player_attributes$ tais que $Team = team$ e $Season = season$

6: $current_team_players \leftarrow$ conjunto de jogadores únicos em $current_team_df$

7: definir coluna de minutagem como $Min.$ ou Min

8: $current_team_roster \leftarrow$ jogadores únicos em $current_team_df$

9: $lowest_minutes_players \leftarrow$ jogadores de $current_team_df$ ordenados por minutagem crescente

Função auxiliar PREVE($new_players$):

10: $n_remove \leftarrow |new_players|$

11: $removed_players \leftarrow$ primeiros n_remove jogadores de $lowest_minutes_players$

12: $simulated_roster \leftarrow$ jogadores de $current_team_roster$ que não pertencem a $removed_players$, unidos a $new_players$

13: **return** PREVE_PONTOS($team, season, simulated_roster, player_attributes, player_attributes_PCA, points$)

14: $base_players \leftarrow$ jogadores únicos em $player_attributes$ com $Season = prev_season$

15: $player_names \leftarrow$ amostra aleatória de até 100 jogadores de $base_players$

16: $price_map \leftarrow$ dicionário que associa cada jogador de $prev_season$ ao seu $Valor_Mercado$

17: $feasible_pool \leftarrow$ jogadores em $player_names$ tais que não pertencem a $current_team_players$ e existem em $price_map$

18: embaralhar $feasible_pool$

```
19:  current_new_players ← ∅
20:  remaining_budget ← budget
21:  for cada jogador p em feasible_pool do
22:      if |current_new_players| ≥ size then
23:          break
24:      end if
25:      p_price ← price_map[p]
26:      if p_price ≤ remaining_budget then
27:          adicionar p em current_new_players
28:          remaining_budget ← remaining_budget − p_price
29:      end if
30:  end for
31:  current_score ← PREVE(current_new_players)
32:  improvement ← verdadeiro
33:  while improvement = verdadeiro do
34:      improvement ← falso
35:      best_score ← current_score
36:      best_players ← nulo
37:      best_budget ← nulo
38:      for i = 1 até |current_new_players| do
39:          for cada jogador p em player_names do
40:              if p ∈ current_new_players then
41:                  continue
42:              end if
43:              if p ∈ current_team_players then
44:                  continue
45:              end if
46:              if p ∉ price_map then
```

```
47:                 continue
48:             end if
49:             p_price ← price_map[p]
50:             old_player ← current_new_players[i]
51:             old_price ← price_map[old_player] ou 0 se ausente
52:             if p_price > remaining_budget + old_price then
53:                 continue
54:             end if
55:             new_players ← cópia de current_new_players
56:             substituir o jogador da posição i por p
57:             new_score ← PREVE(new_players)
58:             if new_score > best_score then
59:                 best_score ← new_score
60:                 best_players ← new_players
61:                 best_budget ← remaining_budget + old_price − p_price
62:             end if
63:         end for
64:     end for
65:     if best_players ≠ nulo then
66:         current_new_players ← best_players
67:         current_score ← best_score
68:         remaining_budget ← best_budget
69:         improvement ← verdadeiro
70:     end if
71: end while
72: return current_new_players, current_score, remaining_budget
73: end function
```